

| 1 数算-绪论

| 1 绪论

| 基本概念

程序=数据结构+算法

计算机的两大应用领域：

- 数值计算
- **非数值计算**（主要）

| 基本术语

| 数据

计算机处理的数值、字符等符号的总称

| 数据元素

数据结构中讨论的基本单位

一个数据元素也可以由一个或若干个数据项组成，数据项是有意义的数据的最小原子单位。

| 数据对象

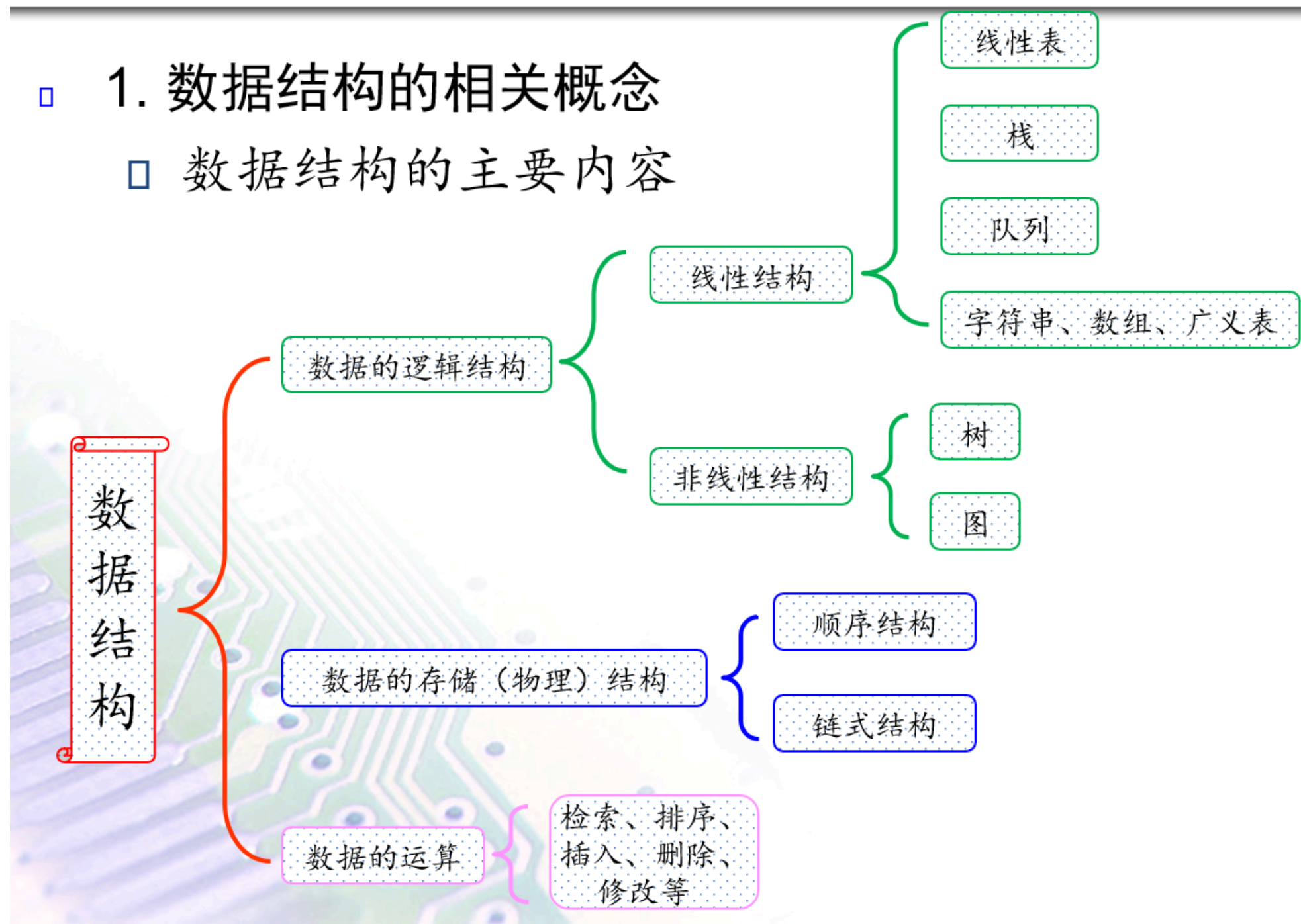
性质相同的数据元素的一个集合

| 数据结构

带结构的数据元素的集合/带结构的数据对象

□ 1. 数据结构的相关概念

□ 数据结构的主要内容



- 逻辑结构
 - 集合结构（元素之间无逻辑关系）
 - 线性结构
 - 非线性结构
 - 树状结构
 - 图状结构
- 物理结构
 - 顺序存储
 - 链接存储
 - 索引存储
 - 散列存储

| 抽象数据类型ADT

数据类型：高级程序设计语言所定义的某类数据的集合和定义在该集合上的一组操作的总称。

| 算法和算法分析

- 伪代码：介于自然语言和计算机语言之间的文字和符号（包括数学符号）来描述算法。
- 算法和程序的区别
 - 算法必须满足有穷性，程序可无穷循环执行
 - 算法是运算步骤的模型，程序指令必须可执行
- 算法的定义
 - 一个有穷的指令集，这些指令为解决某一特定任务规定了一个运算序列。
- 算法的特征

☞ 输入：有0个或多个输入

及伪代码语言，以及自然语言操作命令。

☞ 输出：有一个或多个输出（处理结果）

☞ 确定性：每步定义都是确切、无歧义的

☞ 有穷性：算法应在执行有穷步后结束

☞ 可行性：任何计算步骤都是可以被分解为基本的可执行的操作步，即每个计算步都可以在有限时间内完成。

- 算法的评价标准：
 - 正确
 - 健壮
 - 可读
 - 效率

| 算法的分析

| 时间复杂度

算法所花费的时间代价。

问题的规模以某种单位由1增至n时，对应算法所耗费的时间也以某种单位由f(1)增至f(n)。此时称该算法的时间复杂度为f(n)。

同一个算法，视面对的数据不同，会导致时间代价不同：

- 最好情况下的时间代价

- 平均情况下的时间代价
- **最坏情况下的时间代价**（最常用），用大O法表示 $O((n))$
 - 关注复杂性的数量级，忽略数量级的系数
 - 关注上升最快的一项
 - **循环的时间复杂度等于该循环体的复杂度乘以循环的次数**

| 空间复杂度

所需占用的空间代价

和时间复杂度类似，最坏情况下用 $O(f(n))$ 表示

- 是为解决问题所需要的辅助存储空间
- 只有完成同一功能的几个算法之间才具有可比性
- 忽略零星变量的存储空间
- 没有时间复杂度那么重要

| 常见复杂度排序

◆ 常见的时间/空间复杂度

- ☞ 常数阶 $O(1)$
- ☞ 对数阶 $O(\log_2 n)$
- ☞ 线性阶 $O(n)$
- ☞ 线性对数阶 $O(n \log_2 n)$
- ☞ 平方阶 $O(n^2)$
- ☞ 立方阶 $O(n^3)$
- ☞ k 次方阶 $O(n^k)$ $k > 3$
- ☞ 指数阶 $O(2^n)$ 。

复杂度依次增大

| 作业

| 1

分析递归函数的时间复杂度。简要说明分析过程。

```
int factorial(int n)
{
```



```

    if(n<=0)return 1;
    return n*factorial(n-1);
}

```

设时间复杂度为 $T(n)$ ，则 $T(n)=O(1)+T(n-1)$ ，其中 $O(1)$ 对应的是if判断和 n 乘以 $factorial(n-1)$ 的操作步骤， $factorial(n-1)$ 的时间复杂度为 $T(n-1)$ ，所以 $T(n)=O(1)+T(n-1)=2*O(1)+T(n-2)=\dots=n*O(1)+T(0)=O(n)$

| 解题方法：循环的时间复杂度等于该循环体的复杂度乘以循环的次数

2

设 n 是描述问题规模的非负整数，下列程序段的时间复杂度是多少？简要说明分析过程。

```

x=0;
while(n>=(x+1)*(x+1))
    x++;

```

设时间复杂度为 $T(n)$ ，1行的时间复杂度为 $O(1)$ ，2-3行为while循环，当 $(x+1)*(x+1)$ 大于 n 时，结束循环，其时间复杂度为 $O(n)$ ，所以 $T(n)=O(1)+O(n)=O(n)$

| 解题思路：同上

3

设 n 是描述问题规模的非负整数，下列程序段的时间复杂度是多少？简要说明分析过程。

```

count=0;
for(k=1;k<=n;k*=2)
    for(j=1;j<=n;j++)count++;

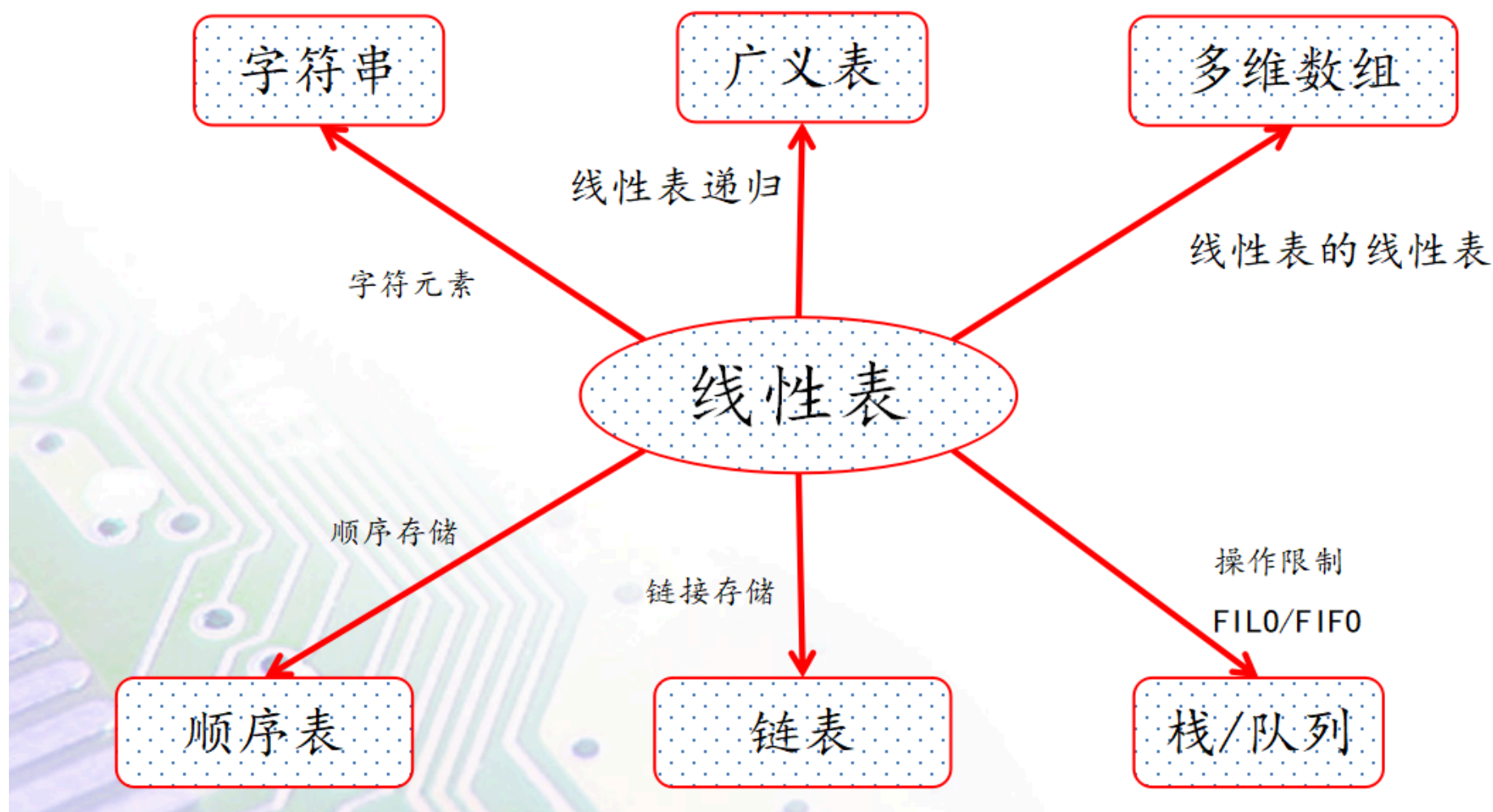
```

设时间复杂度为 $T(n)$ ，1行的时间复杂度为 $O(1)$ ，2行为外层for循环，时间复杂度为 $O(n)$ ，3行为内循环，时间复杂度为 $O(n)$ ，所以 $T(n)=O(1)+O(n)*O(n)=O(n^2)$

| 解题思路：同上

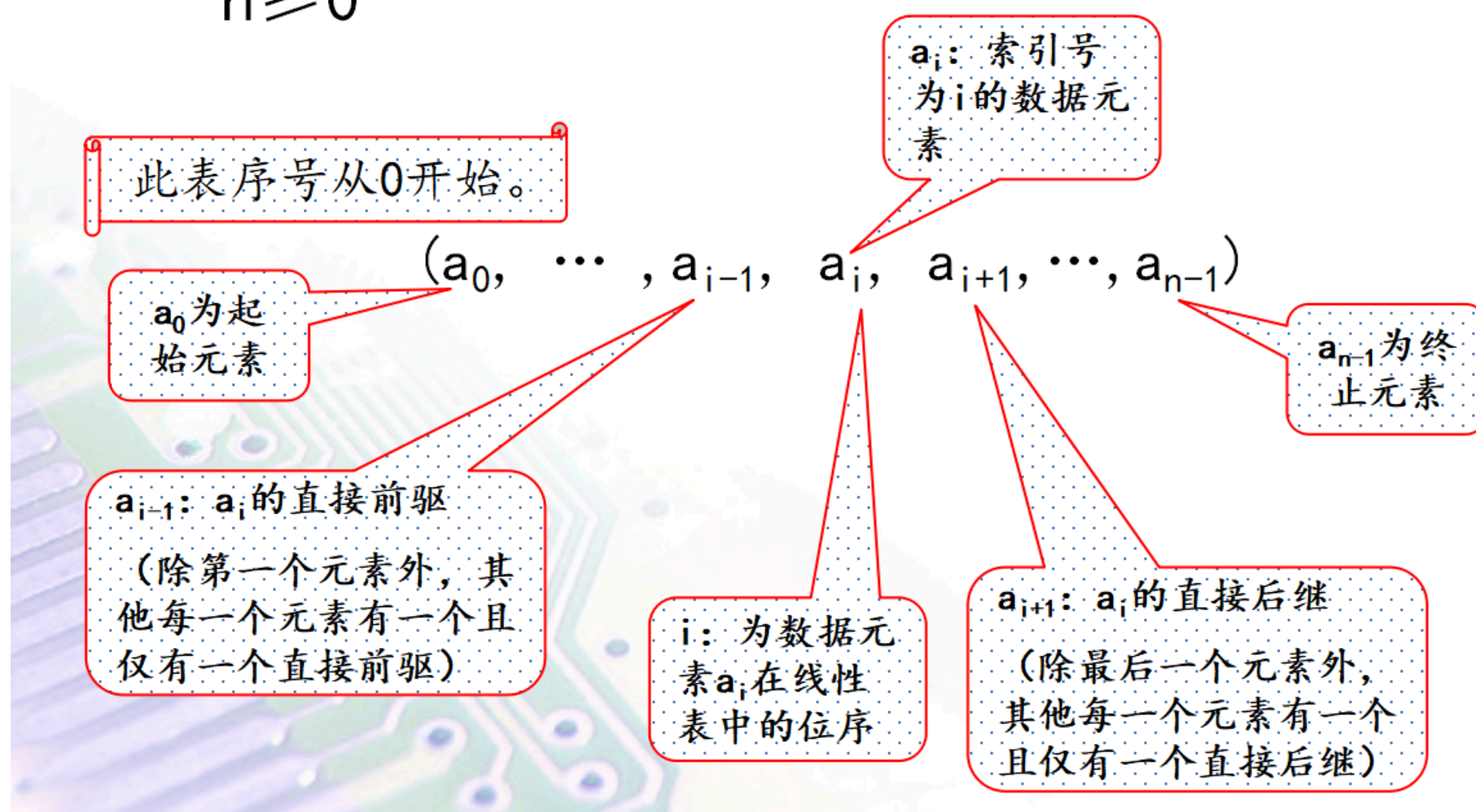
2 线性表

◆ 围绕线性表的课程内容



术语

◆ 线性表中元素的个数 n 定义为线性表的长度， $n \geq 0$



- n 为线性表长度
- i 为位序
- a_0 为起始元素
- a_{n-1} 为终止元素
- a_{i-1} 为 a_i 的直接前驱
- a_{i+1} 为 a_i 的直接后驱

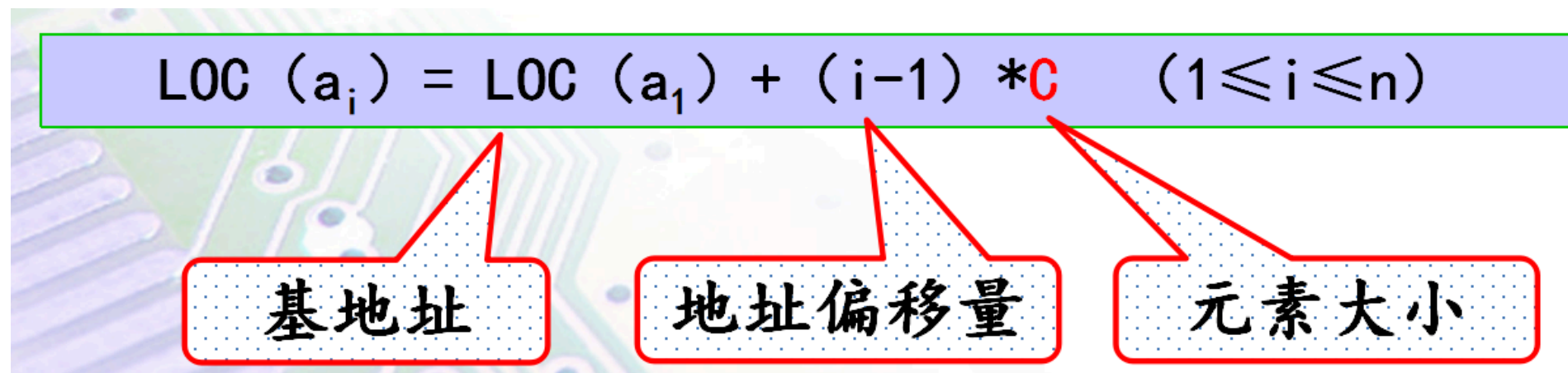
3 顺序表

顺序表是一种线性表，采用顺序存储方法存储的线性表简称为顺序表。

- 顺序存储：把线性表的结点按逻辑次序依次存放在一组地址连续的存储单元里

顺序表的地址顺延，所以对于任意一个元素，都能以 $O(1)$ 找到它的地址：

$$LOC(a_i) = LOC(a_1) + (i - 1) * C$$



优点

存储方式简单，便于随机访问，访问效率高

缺点

扩充、插入和删除操作不方便

操作

插入

当插入元素x条件具备时，将插入位置以及其后的节点依次向后移动一个单元,然后在空出的位置上插入新结点x。

需移动元素个数：n-i
平均需移动元素个数：n/2
时间复杂度： $O(n)$

删除

当删除元素x条件具备时，将删除位置以后到最后元素依次向前移动一个单元,使删除位置元素被覆盖并维持了顺序表的存储方式

需移动元素个数：n-i-1
平均需移动元素个数：n-1/2
时间复杂度： $O(n)$

4 链表

 考点猜测

编程题最大可能来源。

链接方式存储的线性表简称为链表。

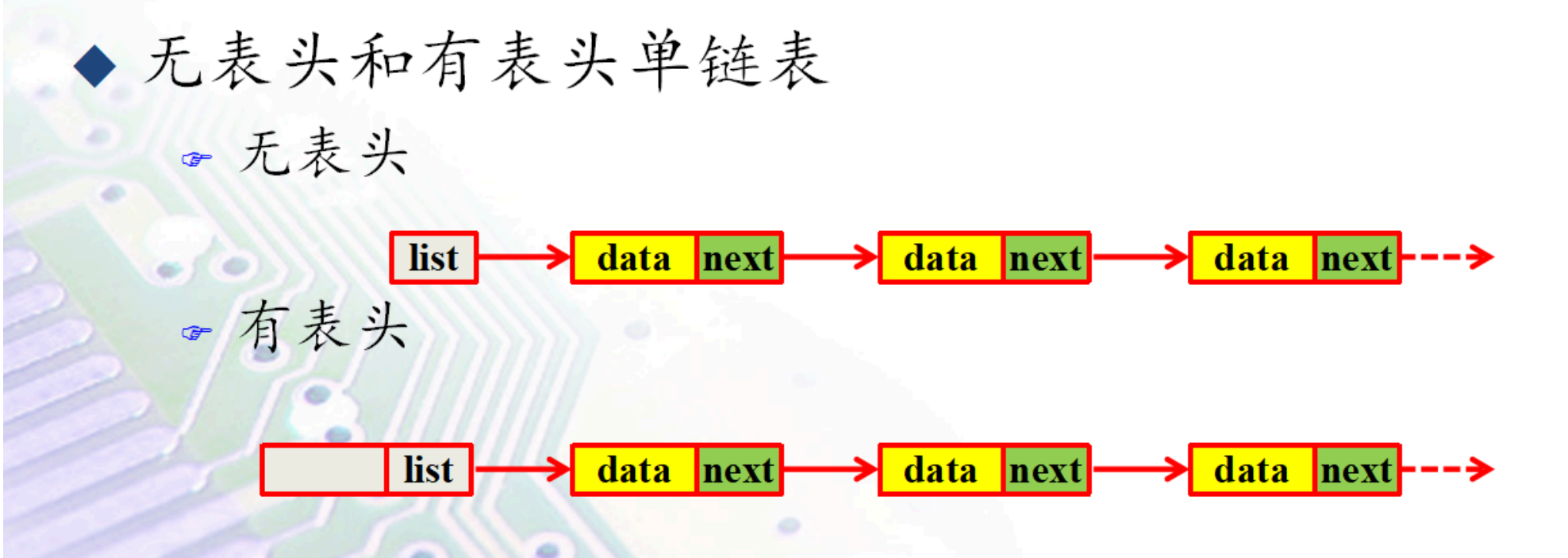
- 用一组任意的存储单元存储表
- 每个数据元素(ai)不但包含元素值(data)，还必须存储相邻数据元素(ai-1)或(ai+1)的地址。

单链表

每个节点包含两个域，data存储该节点的元素，next存储下一节点的地址



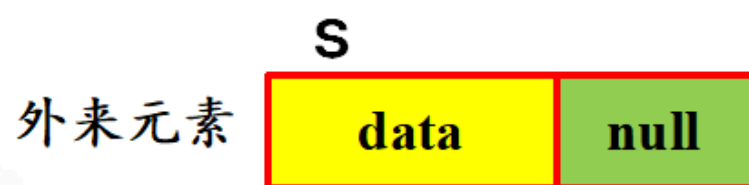
分为有表头和无表头



操作

插入

✎ 插入前



✎ 插入后



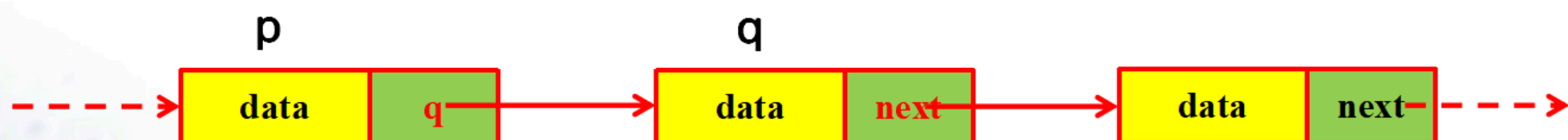
```
s->next = p->next; // 1.s的后继指向p的后继  
p->next = s;       // 2.p的后继指向s
```

时间复杂度： $O(1)$

删除

◆ 节点q的删除

✎ 删除前



✎ 删除后



```
q = p->next; // 1.q指向待删除节点  
p->next = q->next; // 2.p的后继指向q的后继  
free(q); // 3.释放q节点
```


不要忘记释放q节点。
时间复杂度： $O(1)$

无表头单链表

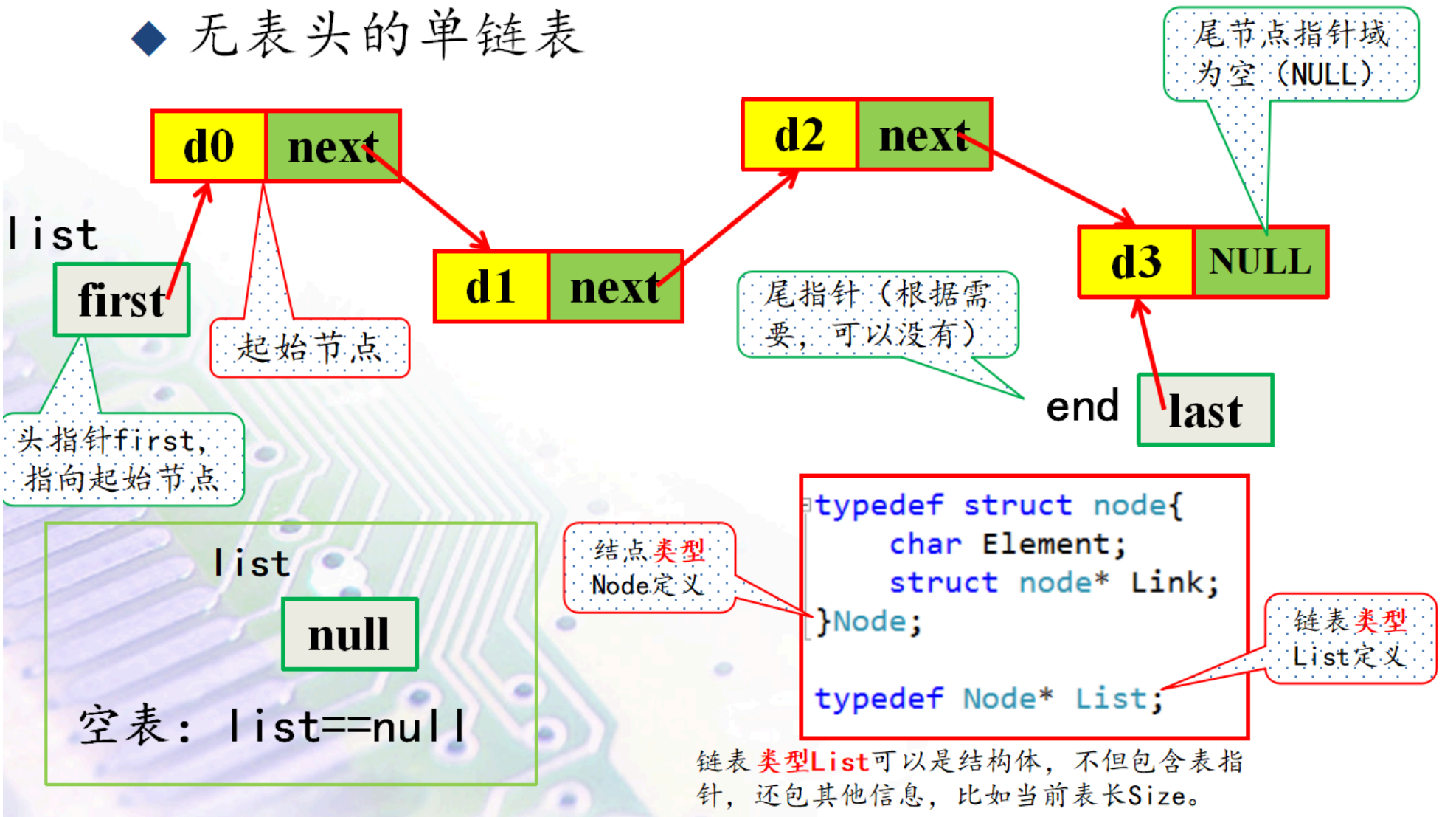
数据结构定义：

```
typedef struct node{
    char Element;
    struct node* Link;
}Node; //节点类型定义
typedef Node* List //头指针定义
List head;
```

空表的时候：`list==null`

■ 单链表(Singly Linked List)的结点结构

◆ 无表头的单链表



头插法

[【链表的头插法，数据结构与算法完整代码动画，考研408 期末考试数据结构】](#)

```
struct Node *head = NULL;
void insertHead(int e){
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = e;
    newNode->next = head;
    head = newNode;
}
```

时间复杂度： $O(1)$

尾插法

```
struct Node* head;
void insertLast(int e){
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = e;
```

```

newNode -> next = NULL;
if(head == NULL){
    head = newNode;
}
else{
    Node* lastNode = head;
    while(lastNode->next != NULL){
        lastNode = lastNode -> next;
    } //寻找尾节点
    lastNode -> next = newNode;
}
}

```

时间复杂度： $O(n)$

有表头单链表

数据结构定义：

```

typedef struct node{
    char Element;
    struct node* Link;
}Node; //节点类型定义
typedef Node* List //头指针定义
List head = (Node*) malloc(sizeof(Node)); //头节点定义
head -> next = NULL;

```

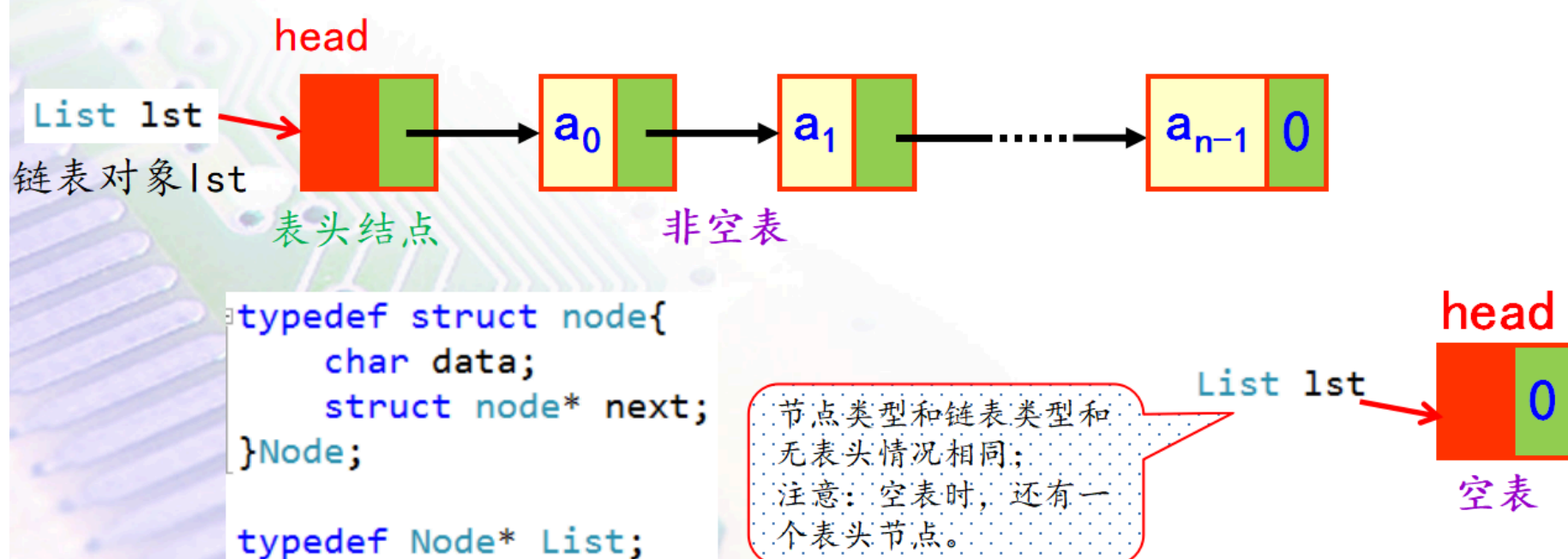
同无表头

表头结点位于表的最前端，本身不带数据

■ 单链表(Singly Linked List)的结点结构

◆ 有表头结点的单链表

- ☞ 表头结点位于表的最前端，本身不带数据。
- ☞ 表头结点的存在统一了空表与非空表的操作，简化了链表设计过程。



表头节点（头节点）后的第一个节点（整个链表的第一个数据节点）称为**首元节点**。

有表头单链表简化在哪里？

无表头单链表需要区分空表时头指针为NULL的情况，而有表头单链表即使是空表，仍然存在头节点，头指针不为空。所以在寻找有表头单链表的最后一个元素时，直接按照判定一个普通节点是否有后继（ $\text{node} \rightarrow \text{next} \neq \text{NULL}$ ）即可。

| 头插法

```
List head = (List)malloc(sizeof(List))
List_Insert(int e){
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = e;
    newNode->next = head->next;
    head->next = newNode;
}
```

时间复杂度： $O(1)$

| 尾插法

这里令头节点中存储链表的长度。

```
List head = (List)malloc(sizeof(List));
head->next = NULL;
head->data = 0;
void insertLast(int e){
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = e;
    newNode->next = NULL;
    Node *lastNode = head;
    while(lastNode->next != NULL){
        lastNode = lastNode->next;
    }
    lastNode->next = newNode;
    head->data++;
}
```

时间复杂度： $O(n)$

| 遍历求表长

```
int GetListLength(List p)
{
    int length = 0;
    while(p->next != NULL){
        p = p->next;
        length++;
    }
    return length;
}
```

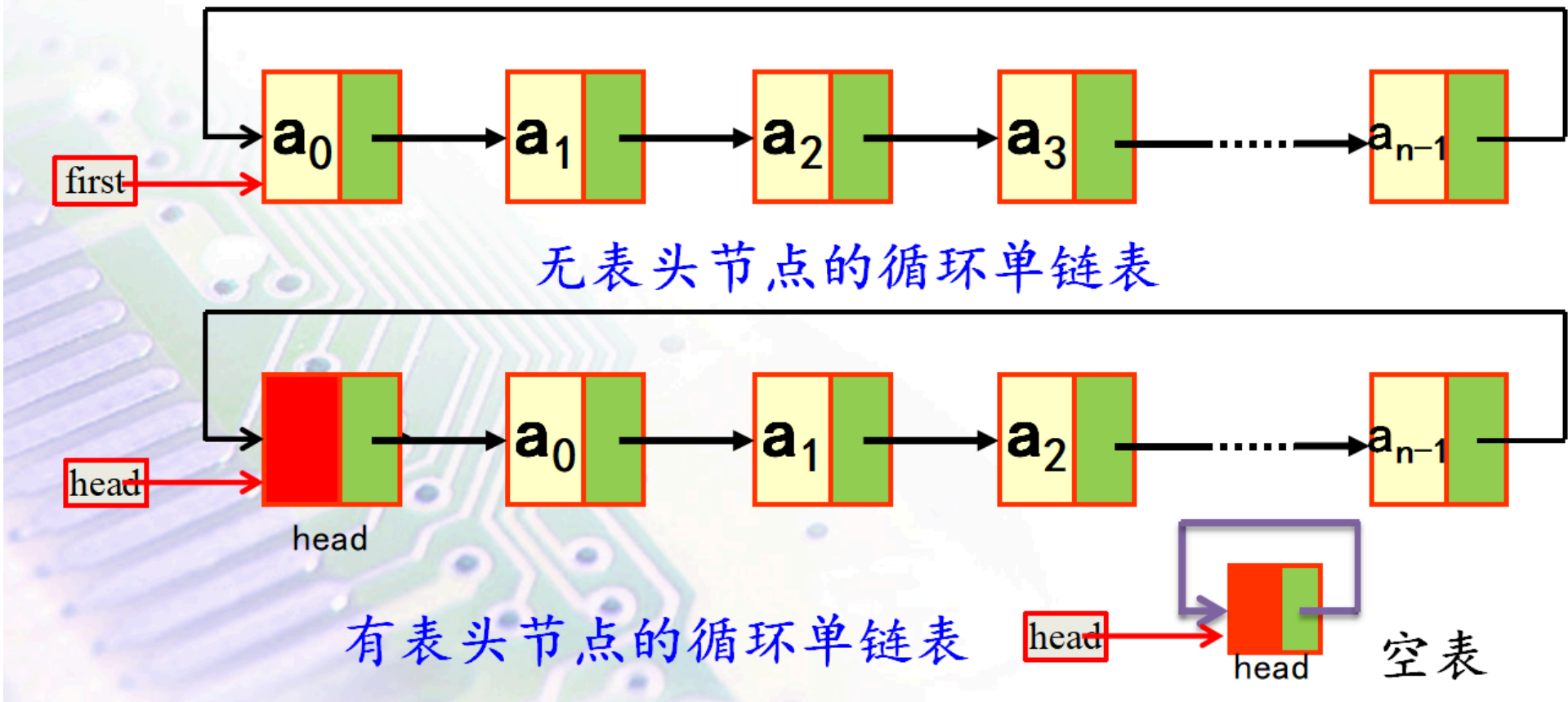
时间复杂度： $O(n)$

| 返回位i的数据

```
char GetListNode(List p,int i){
    int j=0;
    while(p!=NULL && j<i){
        p = p->next;
        j++;
    }
    if(p==NULL || j>i)
        return -1;
    return p->data;
}
```


循环链表

最后一个结点的next指针不为0(NULL)，而是指向了表的前端。从任意结点出发均能找到其他结点。



- 判断空表的条件：
 - `head == head->next`
- 判断表尾节点的条件
 - `head == p->next`

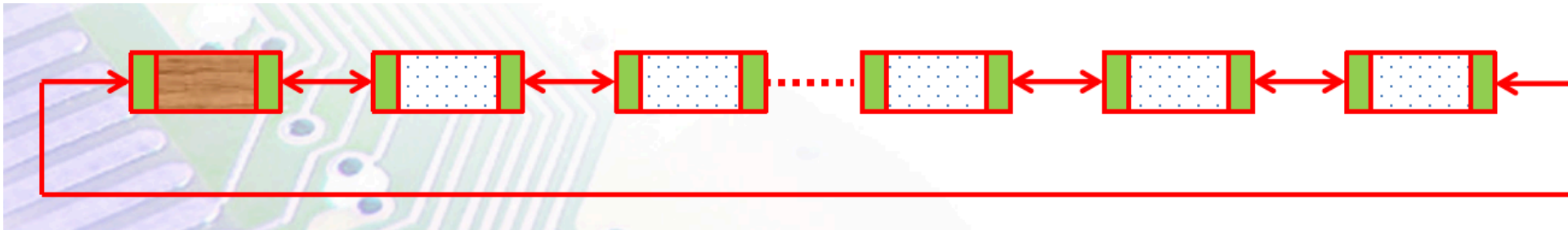
双链表

双链表一般被设计为带表头结点的循环双链表，便于操作

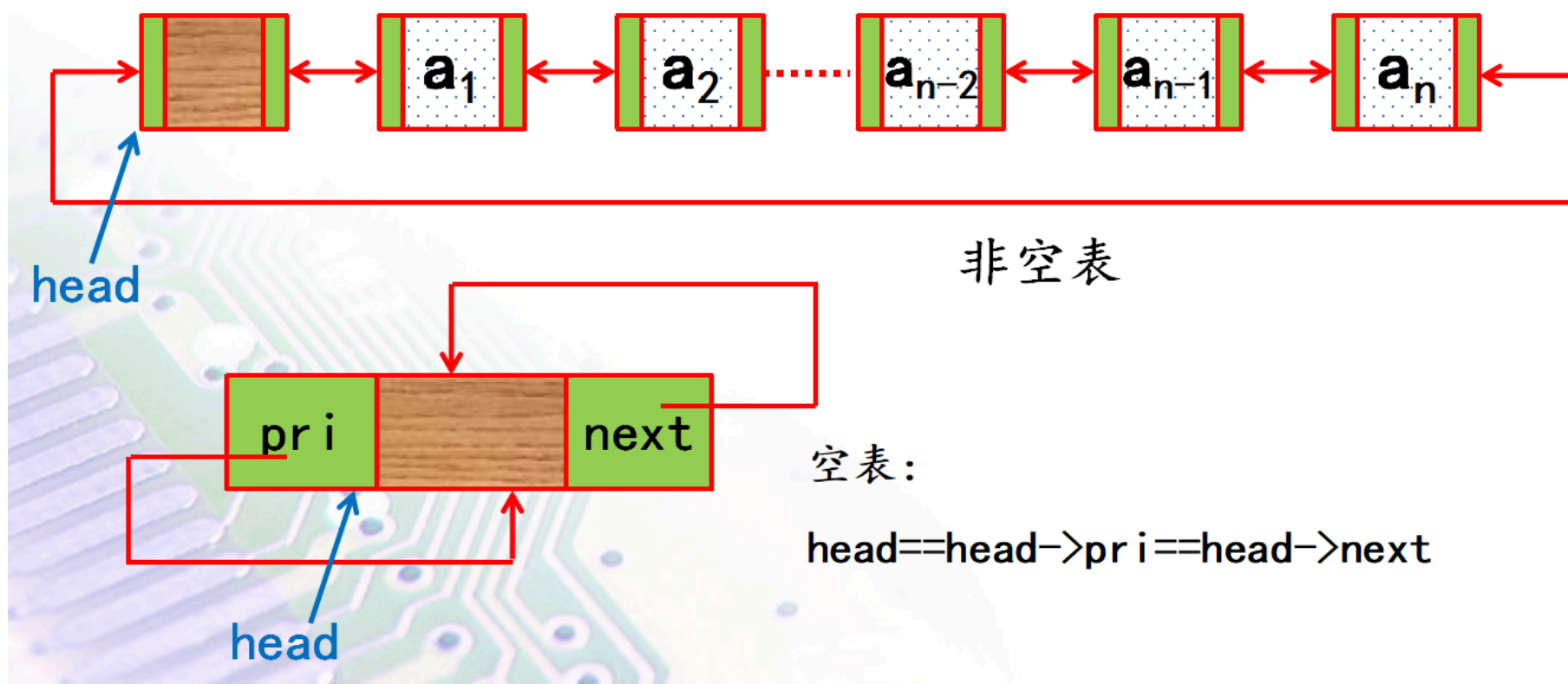
单个节点的结构：



循环双链表结构：



- 判断空表的条件：
 - `head == head->pri == head->next`



数据结构定义：

```
typedef struct node{
    struct node* prior;
    char data;
    struct node* next;
}Node;
```

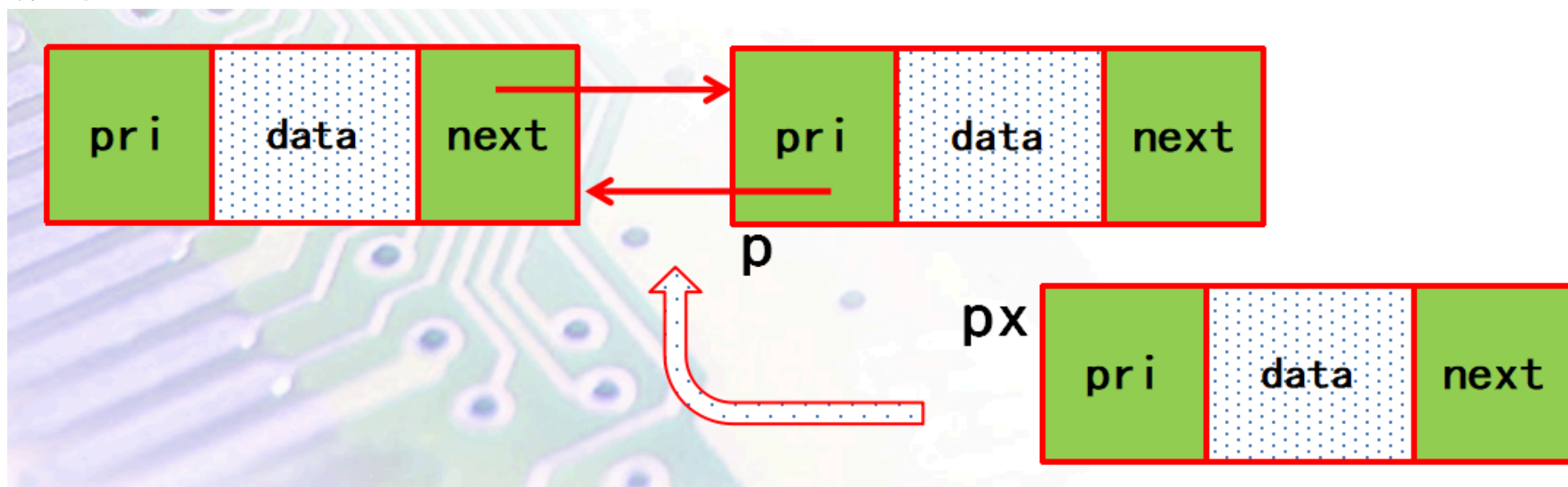
操作

插入

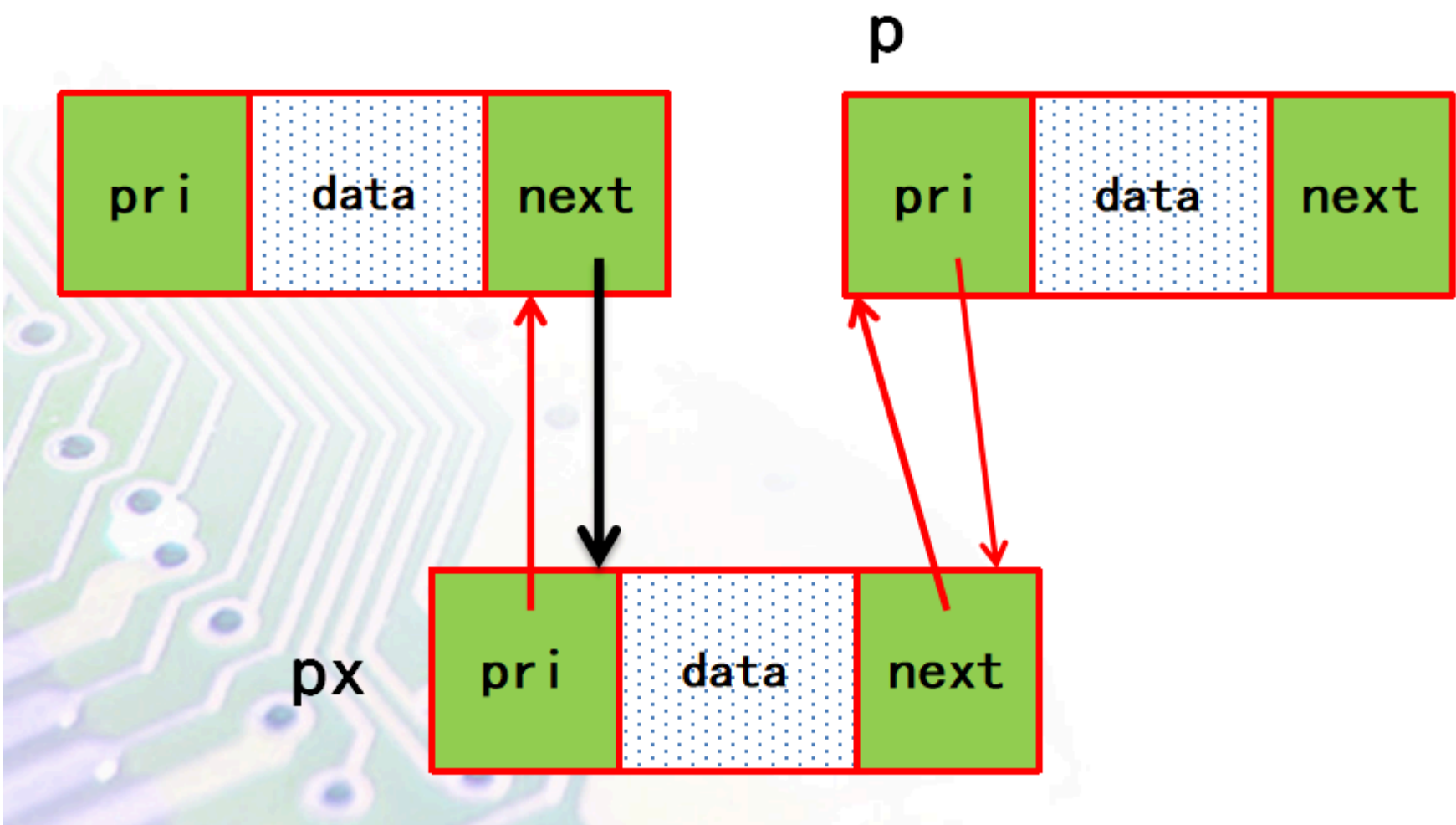
将px节点插到p节点前：

```
px->next=p;           //1. 令px后继指针指向p
px->prior=p->prior;    //2. 令px的前驱指针指向p的前驱
p->prior=px;          //3. 令p的前驱指针指向px
px->prior->next=px;    //4. 令px前驱指针指向px
```

插入前：



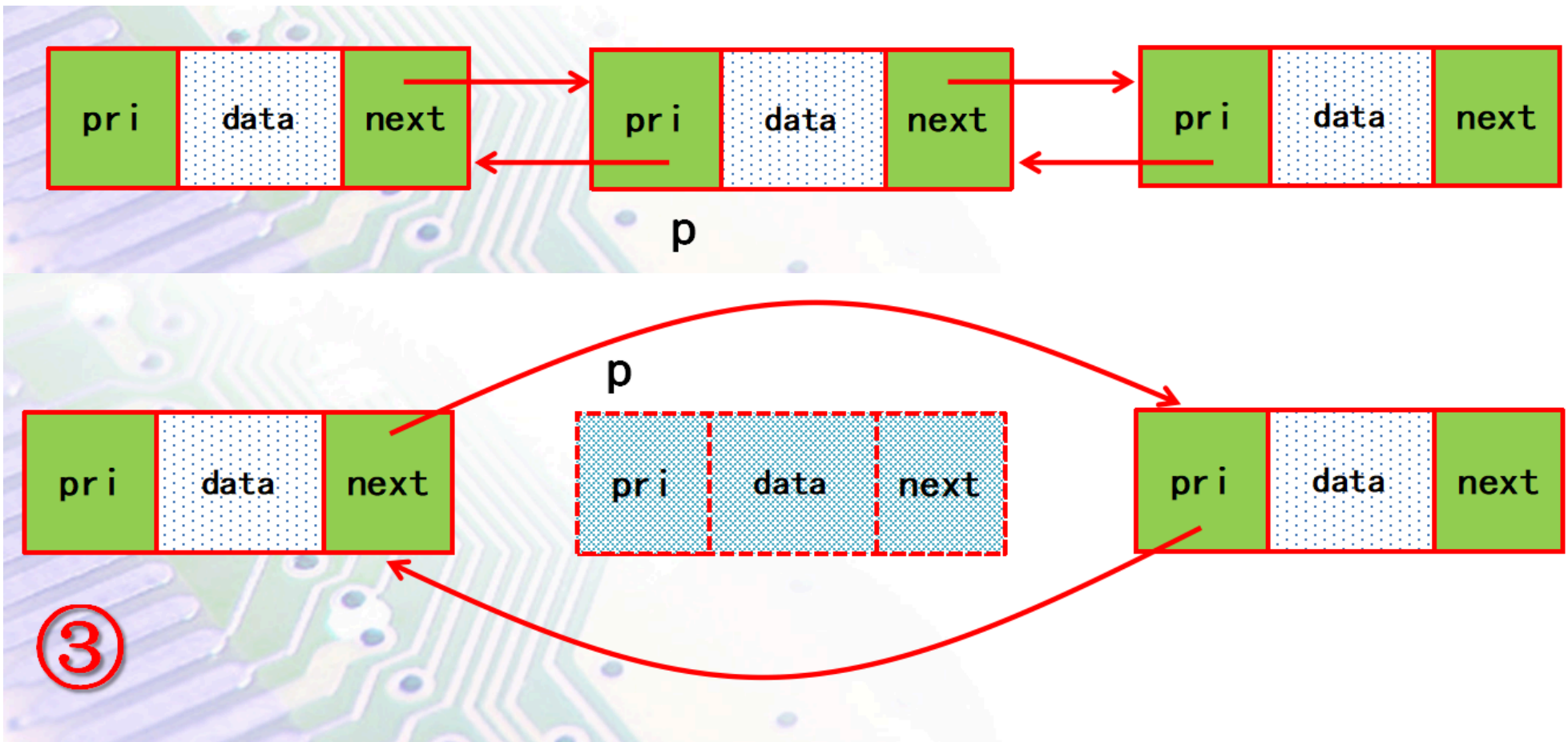
插入后：



删除

将节点p删除：

```
p->next->prior=p->prior; //1. 让p的后继指向p的前驱
p->prior->next=p->next; //2. 让p的前驱指向p的后继
free(p);                 //3. 释放节点p的空间
```



约瑟夫斯问题

问题描述：N 个人围成一圈，从某人开始报数，数到 k 的人出圈，然后从下一个人继续报数，求出圈顺序

使用顺序表实现：

```
void Josephus(List *input_queue, int removeIndex, List *output_queue){
    int out_pos = 0;
    int i = 0;
    while(input_queue->Length)
    {
```



```

    for(i = 0; i<removeIndex-1; i++)
        out_pos = (out_pos+1) %input_queue->Length;
    T_chd output_data;
    Remove(input_queue, out_pos, &output_data);
    Append(output_queue, output_data);
}
}

```

|作业

题目：

令X= (x₁, x₂,...,x_n) 和Y= (y₁, y₂,...,y_m) 是两个带表头的单链表，编写程序合并X和Y并存储在新的带表头节点的双链表M中，

$$M = \begin{cases} x_1, y_1, x_2, y_2, \dots, x_m, y_m, x_{m+1}, \dots, x_n & \text{if}(m \leq n) \\ x_1, y_1, x_2, y_2, \dots, x_n, y_n, y_{n+1}, \dots, y_m & \text{if}(m > n) \end{cases}$$

运行过程中，X和Y通过键盘输入字符构建单链表，合并后打印双链表M的数据。

链表的定义：

```

typedef struct node{
    char data;
    struct node* next;
}Node;

typedef Node* List;

```

```

typedef struct DNode{
    struct DNode* prior;
    char data;
    struct DNode* next;
}DNode;

typedef DNode* DList;

```

主函数：

```

int main()
{
    printf("请输入字符序列x: 以回车结束\n:");
    List list_x = BottomInsertCreateList();
    printf("请输入字符序列y: 以回车结束\n:");
    List list_y = BottomInsertCreateList();
    PrintList(list_x);
    PrintList(list_y);
    DList list_m=combin_list(list_x, list_y);
    PrintDList(list_m);
    return 0;
}

```

节点插入函数：

```

int InsertDNode(DList lst, char x/*插入节点的数据*/, int i/*插入节点的位置*/)

```

请写出combin_list函数。

```

DList combin_list(List x, List y){
    List px = x->next;
    List py = y->next;
    DList list_m = InitDList();
    while(px || py){
        if(px){

```

```

        InsertDNode(list_m, px->data, 0);
        px = px->next;
    }
    if(py){
        InsertDNode(list_m, py->data, 0);
        py = py->next;
    }
}
return list_m;
}

```

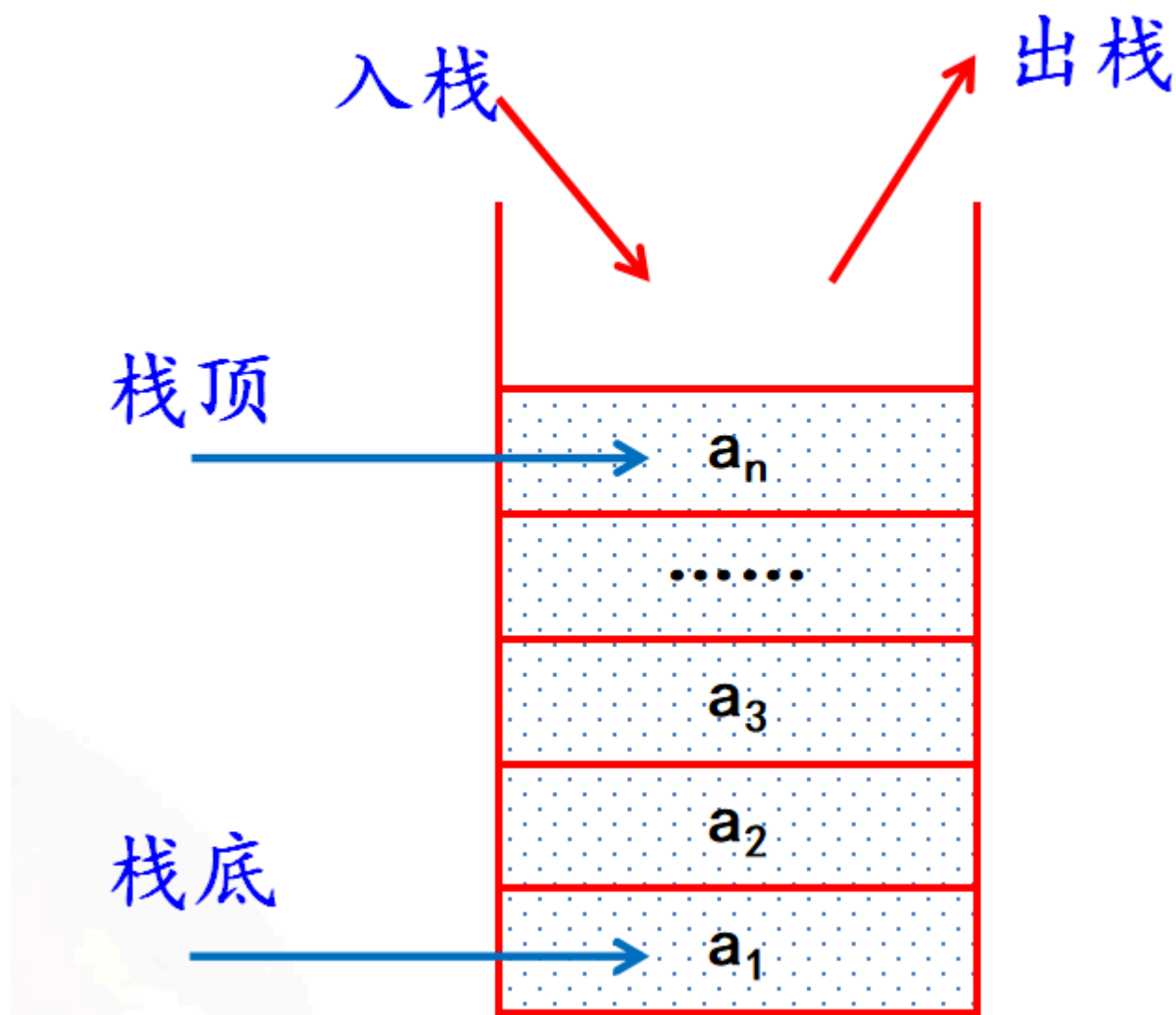
5 栈

🔗 考点猜测

应该重概念，和表达式变化、利用栈计算式子这一块。

只允许在一端插入和删除的线性表

- 具有**后进先出（LIFO）**特性
- 栈顶（top）、栈底（bottom）



- ADT中可以进行的操作：
 - void CreatStack(Stack *s*, int maxsize) \ 栈初始化，构造一个空栈。
 - BOOL IsEmpty(Stack *ps*) \ 栈判空
 - BOOL IsFull(Stack *ps*) \ 栈判满
 - BOOL Push(Stack *ps*, T_chd *px*) \ 入栈
 - BOOL Pop(Stack *ps*, T_chd* *px*) \ 出栈

顺序栈

栈的顺序存储结构简称为顺序栈,是运算受限的顺序表。

- 元素用数组存放，栈底位置是固定不变的，栈顶位置是随着入栈和出栈变化的
- 编程要注意上溢和下溢的处理

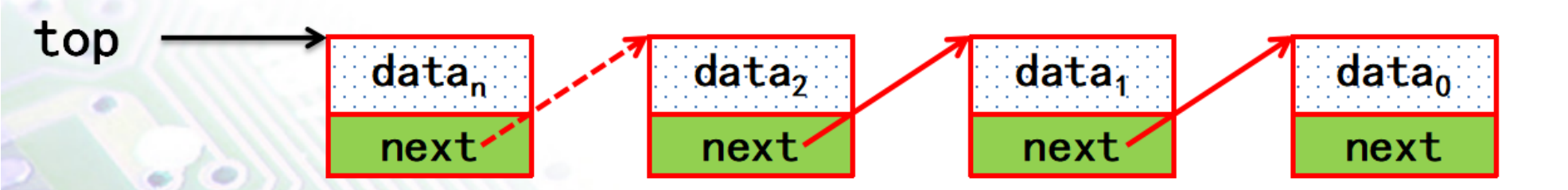
- 上溢 (Overflow): 过度Push
- 下溢 (Underflow): 过度Pop

数据结构类型定义:

```
typedef struct stack{
    int Top;
    int MaxStack;
    T_chd Elements[MAX_SIZE];
}Stack;
```

链栈

栈的链式存储结构称为链栈。是运算受限的链表。



- 链式栈(理论上)无栈满问题，空间可扩充
- 链式栈的栈顶在链头
- 动态分配栈元素，适合于多个栈共享内存，互不干扰

入栈：头插法
出栈：删除首个元素

对比维度		顺序栈	链式栈
时间效率	入栈 (Push)	O(1) (直接通过栈顶指针修改)	O(1) (仅修改头节点指针)
	出栈 (Pop)	O(1) (直接通过栈顶指针修改)	O(1) (仅修改头节点指针)
	读取栈顶元素	O(1) (直接访问栈顶指针指向的元素)	O(1) (直接访问头节点的数据域)
	读取第k个元素	O(1) (通过栈顶位置相对位移直接访问，如数组下标计算)	O(k) (需从头节点开始沿指针链遍历)
空间效率	内存分配	需预先分配固定大小的连续内存 (可能浪费空间或溢出)	动态分配节点内存 (按需增长，无固定大小限制)
	结构性开销	无额外开销 (仅存储数据元素)	每个节点需存储指针 (增加内存占用，如32位系统下每个指针占4字节)
	实际应用	更广泛 (如函数调用栈、系统栈等需要快速定位的场景)	特定场景 (如多栈共享空间、栈大小不确定或频繁动态扩展的场景)
优势场景		1. 栈大小已知且固定 2. 需要频繁读取内部元素 (如第k个元素) 3. 追求低内存开销	1. 栈大小不确定 2. 多栈共享内存空间 3. 动态扩展需求高

应用

数制转换

算法步骤

1. 初始化空栈：创建一个用于存储余数的栈
2. 连续除法求余：
 - 将十进制数N不断除以目标进制d
 - 将每次的余数压入栈中

- 用商替换N继续上述过程，直到商为0

3. 逆序输出结果：将栈中的余数依次弹出，得到转换后的结果

代码示例

把N转为八进制：

```
while(N) {
    Push(S, N%8);
    N = N/8;
}
while(!IsEmpty(S)) {
    Pop(S, e);
    printf("%d", e);
}
```

括号匹配的检验

建立左括号栈，出现右括号出栈一次，“抵消完”即括号匹配

表达式求值

表达式分类：

- 前缀表达式（逆波兰式），不常用
- 中缀表达式：如 $12*(20-5)+30$
 - 符合人类阅读习惯，但计算机处理复杂
- 后缀表达式（波兰式）如： $12\ 20\ 5\ -\ *\ 30\ +$
 - 已隐含了运算优先级，括号不再出现

借助栈实现计算过程，分两步：

- 中缀表达式转后缀表达式
- 算后缀表达式的值

中缀表达式转后缀表达式

从左到右扫描中缀表达式：

- 遇到操作数：直接输出
- 遇到“(": 入栈“("
- 遇到“)”: 持续出栈并输出,直到出栈1个“("。(括号不打印输出)
- 遇到符号：判断该符号和栈顶符号优先级，如果栈顶优先级高，则持续出栈直到栈顶符号优先级小于该符号、或栈顶符号为空或“(", 将该符号入栈
- 扫描完中缀表达式后将栈内剩余符号全部出栈输出

后缀表达式求值

从左到右扫描后缀式，遇到数字立即入栈，遇到符号则连续出栈两个数作为其右操作数和左操作数并将结果入栈，直到扫描结束，结果在栈底出栈即可。

递归

直接或间接的调用自身的过程叫递归过程（recursion）

一个过程或函数在其中直接或间接地调用自身的方法

两大要素：

- 递归要素
 - 问题要能够分解为更小的子问题，并调用自身解决
- 基准要素

- 递归最终要能够结束，防止无限循环。即子问题要能够缩小到最终能得到结果的情形

递归方法进行进制转换：

```
void Convert(int N) {
    if(N) {
        Convert(N/8); // 先递归处理高位
        printf("%d", N%8); // 后输出当前位
    }
}
```

6 队列

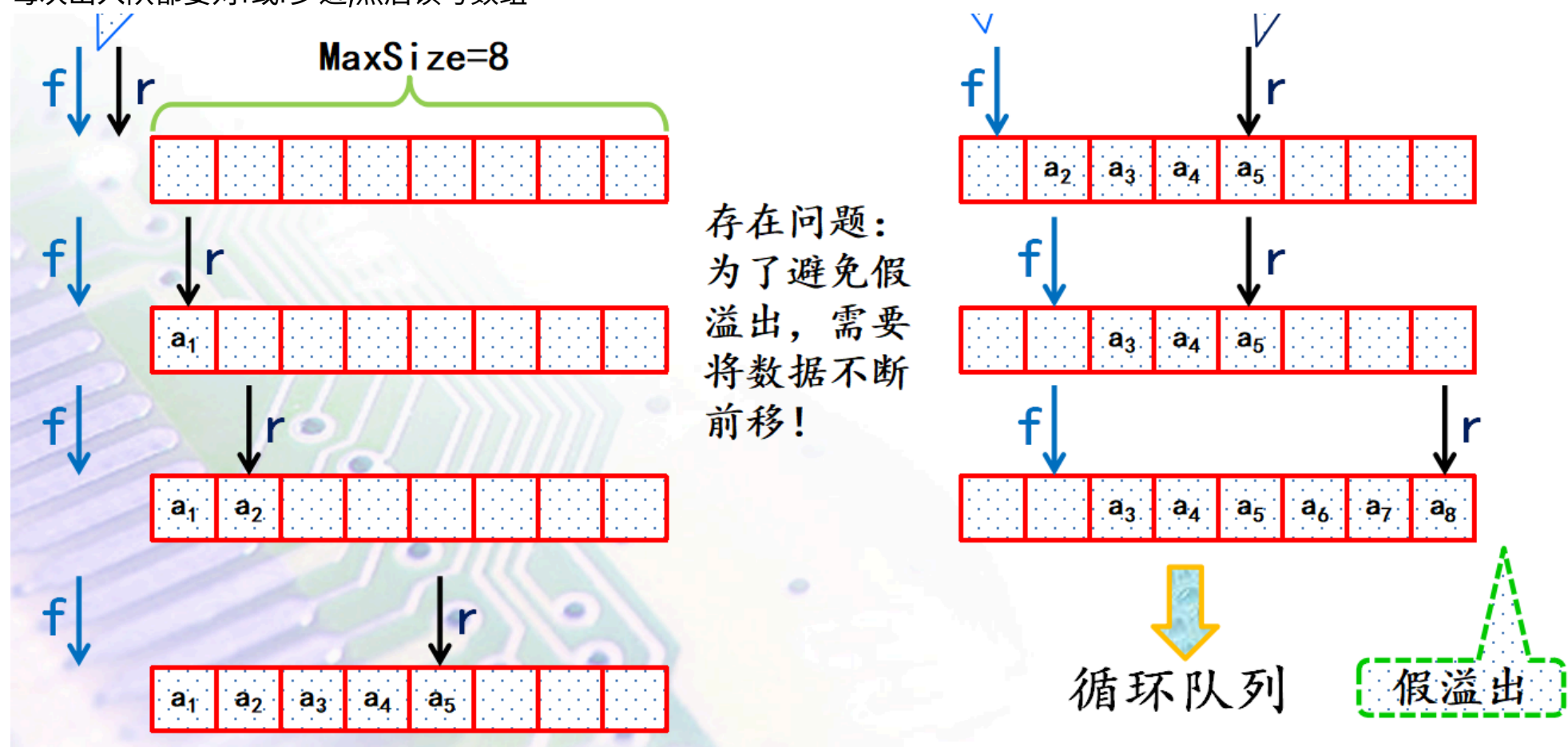
队列是只允许在一端删除，在另一端插入的线性表

- 顺序队列
- 链式队列
- 允许删除的一端叫做队头(front)，允许插入的一端叫做队尾(rear)
- **先进先出 FIFO**特性

顺序队列

f和r的操作要点：

每次出入队都要对f或r步进,然后读写数组

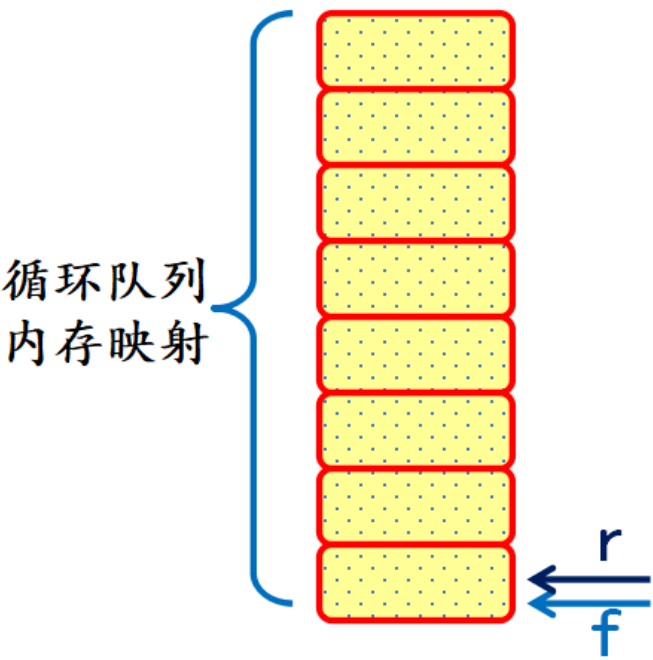
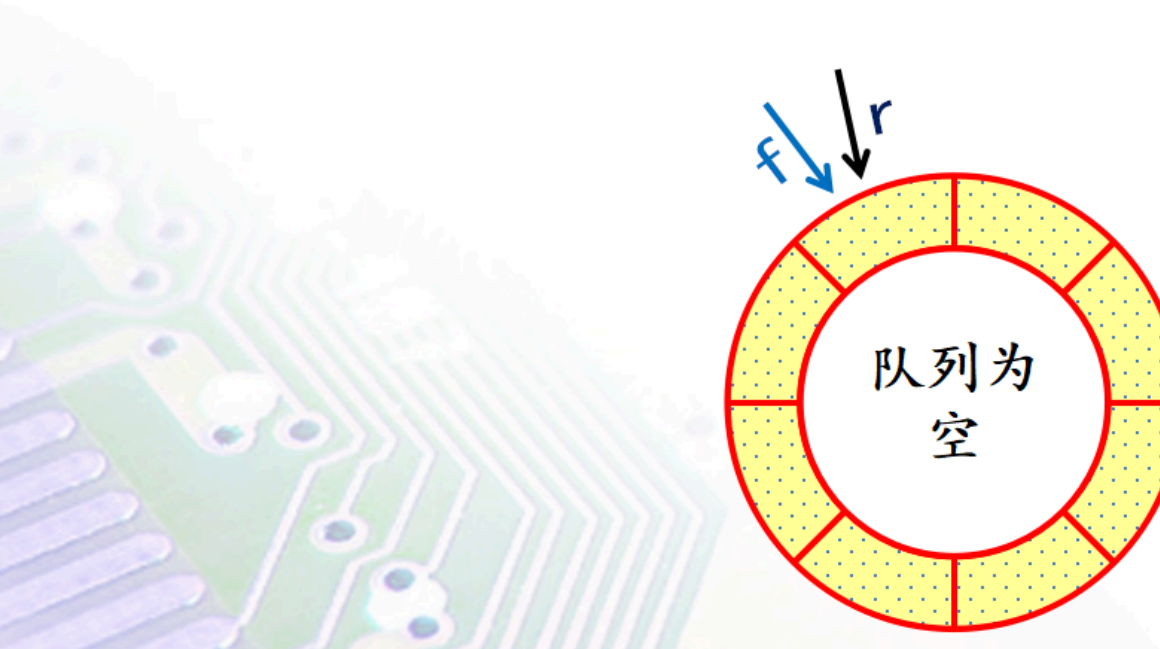


队空判定： `front == rear`

循环顺序队列

■ 队列存储结构

◆ 循环顺序队列



队空判定: $front == rear$

队满判定: $(rear + 1) \% MaxSize == front$ (牺牲一个单元)

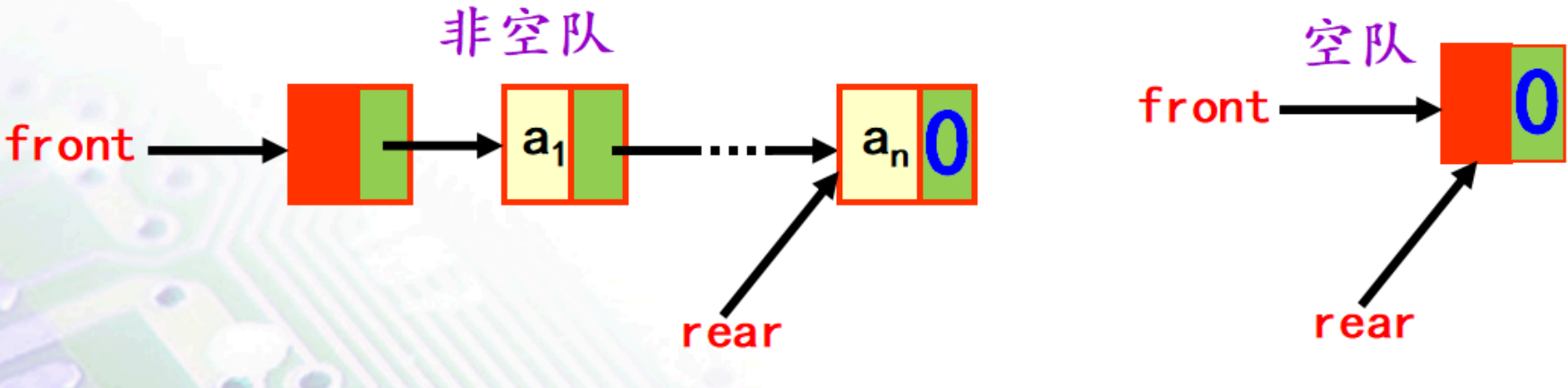
入队: $rear = (rear + 1) \% MaxSize; Q[rear] = x;$

出队: $front = (front + 1) \% MaxSize; x = Q[front];$

队列长度: $(rear - front + MaxSize) \% MaxSize$

链式队列

队头在头结点后，队尾在链尾



队空条件: $front \rightarrow next == NULL$

特性	顺序队列	链式队列
存储结构	固定大小数组	动态链表节点
空间利用率	可能浪费或不足	按需分配，高效
扩容开销	高（需拷贝数据）	无（动态申请节点）
操作效率	$O(1)$ （需处理循环逻辑 $\%size$ ）	$O(1)$ （逻辑简单）
适用场景	已知容量、内存连续性要求高	长度变化大、无法预估容量

应用

解决约瑟夫斯问题

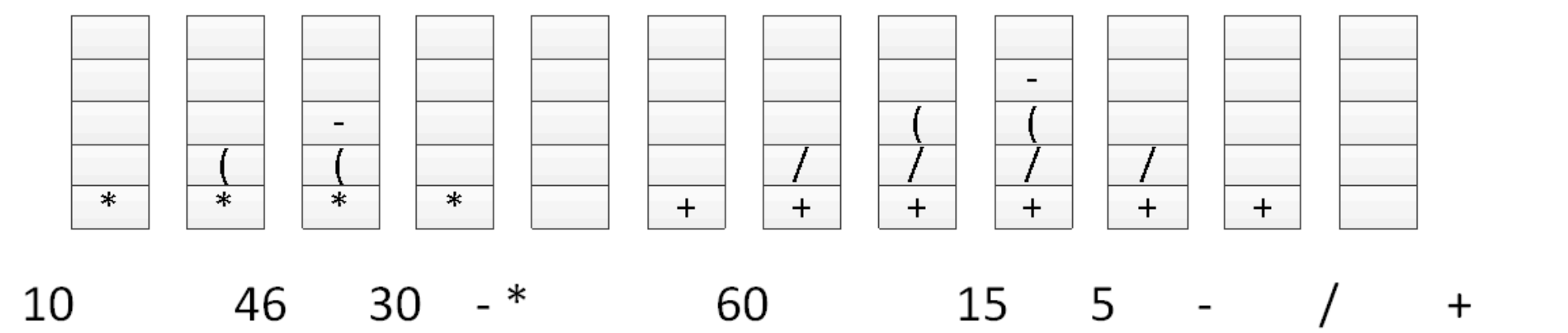
方法：
(1)把队列最前面的元素移到最后一个位置（出队;入队），执行K-1次
(2)队列中的第一元素出队

```
T_chd e, y;  
Dequeue(&qCHD,&e);Enqueue(&qCHD,e); //执行k-1次  
Dequeue(&qCHD,&y); //执行1次
```

作业

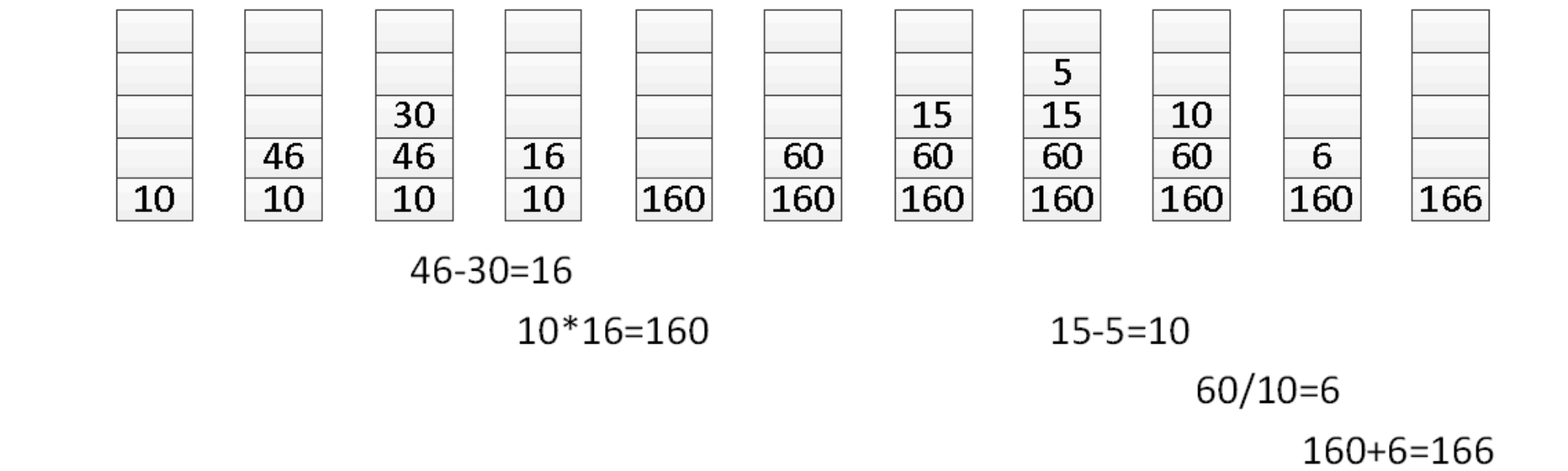
1

请通过栈操作将以下的中缀表达式转换为后缀表达式，并画出栈中内容的变化过程。
 $10 * (46 - 30) + 60 / (15 - 5)$



2

通过栈操作对第1题中的后缀表达式进行求值，并画出栈中内容的变化过程。



所以，结果为：166

7 字符串

字符串

字符串，简称串。由n(n≥0)个字符组成的有限序列

串的大小比较

根据ASCII码，依次比较每一个字符。

与线性表的关系

- 字符串属于线性表结构的特例
- 大多数线性结构以“单个元素”作为操作对象，如查找、插入、删除一个元素；串的操作通常以“连续多个元素”作为操作对象。如查找某个子串、插入一个子串以及删除一个子串。

存储结构

顺序存储

- 紧缩格式
 - 一个字节存放一个字符
- 非紧缩格式
 - 一个字存储单元中放一个字符

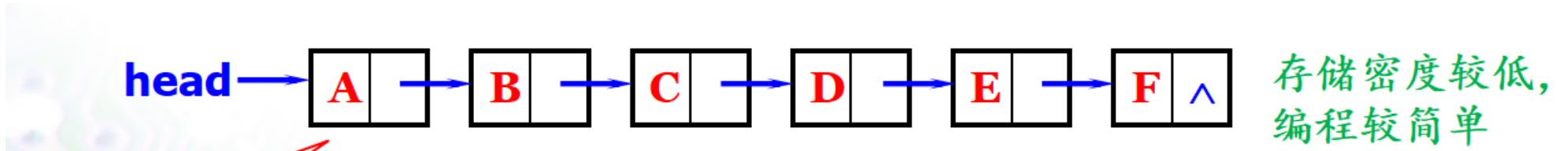
d	a	t	a
\0	字节	字节	字节

紧缩格式存储

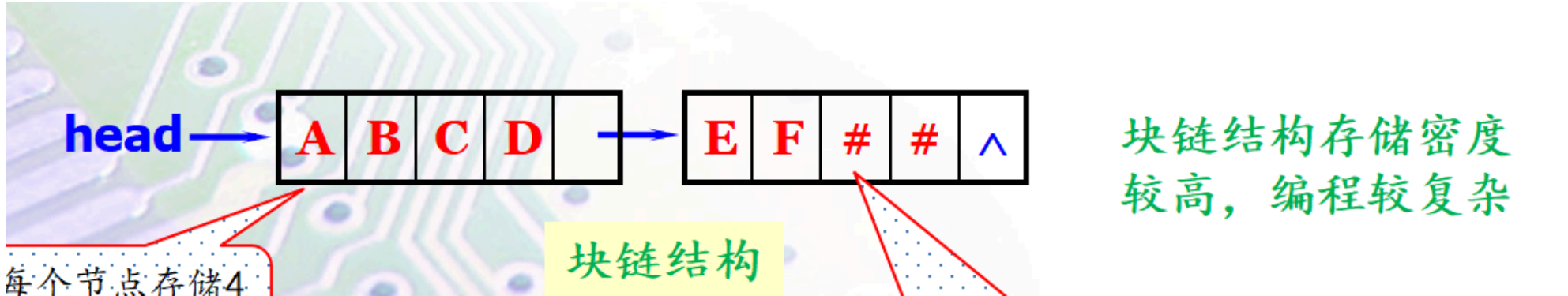
d			
a			
t			
a			
\0			

非紧缩格式存储

链式存储



更复杂的块链结构：



模式匹配Pattern Matching

设S（目标串/主串）和P（模式串）是两个给定的串，在S中找到等于P的子串的过程称为模式匹配。

蛮力(Brute-Force)匹配法

比较 S[i]与 P[j]：

- 若相等，则同时递增 i和 j，继续比较下一个字符
- 若不相等，则主串指针回溯到本次匹配起始位置的下一个位置（即 i=i-j+1），模式串指针重置为 0。

缺点：

- 耗时很大，时间复杂度高，时间复杂度为 $O(m * n)$

| KMP匹配法

参考视频

关键：

- 提前获取最大公共前后缀真子串（next[]数组）

运行过程：

- 主函数中，匹配失败后，不是像蛮力法一样把主指针回溯到开始匹配的第一个字符接着往下对比，而是看next数组，next数组的值是多少就从开始匹配的第一个字符直接向后挪几位，再继续对比
- next数组：最大公共前后缀真子串的长度
 - 写的话直接看前后缀公共长度多长就行，和编程方式差异较大
 - 每个点之前子串中最大的公共前后缀真子串长度
- nextval数组：next的改进版本，规避了重复比较的一些情况，作用不变，但一些情况下更快一点
 - nextval[j]计算方法：nextval[0]=-1，如果t[j]不等于t[next[j]]，nextval[j]=next[j]，否则nextval[j]=nextval[next[j]]

next数组计算：

```
/*计算模式串pCHD的next数组*/
pCHD->next[j = 0] = k = -1; //初始化当前（第0个字符）字符的下标和next值
while (j < pCHD->Length) //循环条件是字符下标没有越界j=0~pp_str->Length-1
{ //从已有的next[j]推算next[j++]
    if (k == -1 || pCHD->chs[j] == pCHD->chs[k])
    {
        //如果当前位置字符比较成功，下一个next自然推进
        j++; k++;
        pCHD->next[j] = k; //写入推进后的k到next[j]
    }
    else //如果不相等，停止步进j，将k作为比较失败位置，查看k之前的子列
        k = pCHD->next[k];
}
```

- 若比较成功→jk都前推，next设为k值
- 若比较失败，k回溯，使用next[k]的值

nextval数组计算：

相比较于next的计算，其实变动点只有匹配成功里面，如果下一个字符也成功，也就是有重复，那么直接使用前面的nextval。

```
/*计算模式串pCHD改进后的nextval数组*/
pCHD->nextval[j = 0] = k = -1; //初始化当前（第0个字符）字符的下标和next值
while (j < pCHD->Length) //循环条件是字符下标没有越界j=0~pp_str->Length-1
{ //从已有的nextval[j]推算nextval[j++]
    if (k == -1 || pCHD->chs[j] == pCHD->chs[k])
    {
        //如果当前位置字符比较成功，下一个next自然推进
        j++; k++;
        //如果产生重复，使用之前的nextval
        if(pCHD->chs[j] == pCHD->chs[k])
            pCHD->nextval[j] = pCHD->nextval[k];
        else
            pCHD->nextval[j] = k;
    }
    else //如果不相等，停止步进j，将k作为比较失败位置，查看k之前的子列
        k = pCHD->nextval[k];
}
```

KMP算法匹配：


```
int/*返回匹配位置*/ KMPIIndex(StringCHD* ps_str/*主串*/,
                             PatternStringCHD* pp_str/*模式串(字串)*/,
                             int pos/*从pos处开始搜索*/,
                             int mode/*0或1:普通失败函数next[j]或改进失败函数nextval[j]*/) {
    int i = pos;    //目标串的初始对比下标
    int j = 0;      //模式串的初始对比下标
    //检查输入参数的合法性：串非空、位置合适

    if (pp_str->Length == 0){
        printf("模式串不能为空\n");
        return -1;
    }
    if (pos < 0 || pos >= ps_str->Length){
        printf("下标越界错误!0~%d", ps_str->Length - 1);
        return -2;
    }

    while (i<ps_str->Length && j<pp_str->Length){
        if (j == -1 || ps_str->chs[i] == pp_str->chs[j]){//字符相同，顺利推进
            i++; j++;
        }
        else//比较失败，做模式串的位置回溯，目标串无需回溯
            j = (mode == 0 ? pp_str->next[j] : pp_str->nextval[j]);
    }
    return (j == pp_str->Length) ? i - pp_str->Length
        /*达到模式串末尾而退出的循环，匹配成功，返回位置*/
        :-3/*未达到模式串末尾而退出的循环，匹配失败*/;
}
```

| 作业

| 1

设模式串t为"abacabaaad "，填写其next数组和改进nextval数组。

j	0	1	2	3	4	5	6	7	8	9
t	a	b	a	c	a	b	a	a	a	d
next[j]	-1	0	0	1	0	1	2	3	1	1
<u>nextval[j]</u>	-1	0	-1	1	-1	0	-1	3	1	1

j	0	1	2	3	4	5	6	7	8	9
t	a	b	a	c	a	b	a	a	a	d
next[j]	-1	0	0	1	0	1	2	3	1	1
nextval[j]	-1	0	-1	1	-1	0	-1	3	1	1

| 2

通过栈S将输入整数转换N为8进制表示输出的核心代码如下：

```
while(N){ Push(S, N%8); N=N/8; }
while(!IsEmpty(S)) { Pop(S,e); printf("%d ",e); }
```

请编写函数void Convert(int)采用递归的方式完成这种输出，Convert函数输入为待转换的整数。说明设计思想。

```
void Convert(int N){
    if(N){
```

```

        Convert(N/8);
        printf("%d ", N%8);
    }
}

```

| 3

已知数组int a[N]，设计递归函数：

(1) int find(int* pA, int len, int x)查找x所在的下标并返回，若无此元素，返回-1.

```

int find(int* pA, int len, int x)
{
    if (pA[--len] == x) return len;
    if (len == 0) return -1;
    return find(pA, len, x);
}

```

(2) int get_max(int* pA, int len)求数组的最大值。

```

int get_max(int* pA, int len)
{
    if (len == 0) return pA[0];
    int m = get_max(pA, --len);
    return m > pA[len] ? m : pA[len];
}

```

(3) double average(int* pA, int len)求数组的平均值。

```

double average(int* pA, int len)
{
    if (len == 0) return pA[0];
    return (average(pA, --len) * len + pA[len]) / (len + 1);
}

```

| 8 稀疏矩阵

当一个阶数较大的矩阵中非零元素个数 s 相对于矩阵元素总个数 t 十分小时 ($s \ll t$)，称该矩阵为稀疏矩阵。

相比直接采用数组存储，可以只存储非零元素，采用三元组 **<行号，列号，数据>** 实现空间的压缩

| 快速转置

[参考视频](#)

稀疏矩阵存储元素是按行优先存储的，保证顺序，转置的难点其实不是对于单个元素的转置，这只要把行列互换一下就可以了，问题是转置之后元素表也要按行优先。

因此需要**列位置偏移表**（CPOT），存储了对应列中第一个非零元素在目标三元组表的下标。当然在此之前需要统计转置前每一列的非零元素个数。

| 9 树和二叉树

| 树

定义：

- 由 $n(n \geq 0)$ 个结点组成的有限集合
- 当 $n \geq 1$ 时，有且仅有一个特定的称为**根**的节点
- 除根以外的其它结点划分为 $m(m \geq 0)$ 个互不相交的有限集合 $T_0, T_1, \dots, T_{m-1}$ ，每个集合又是一棵树，并且称之为根的子树

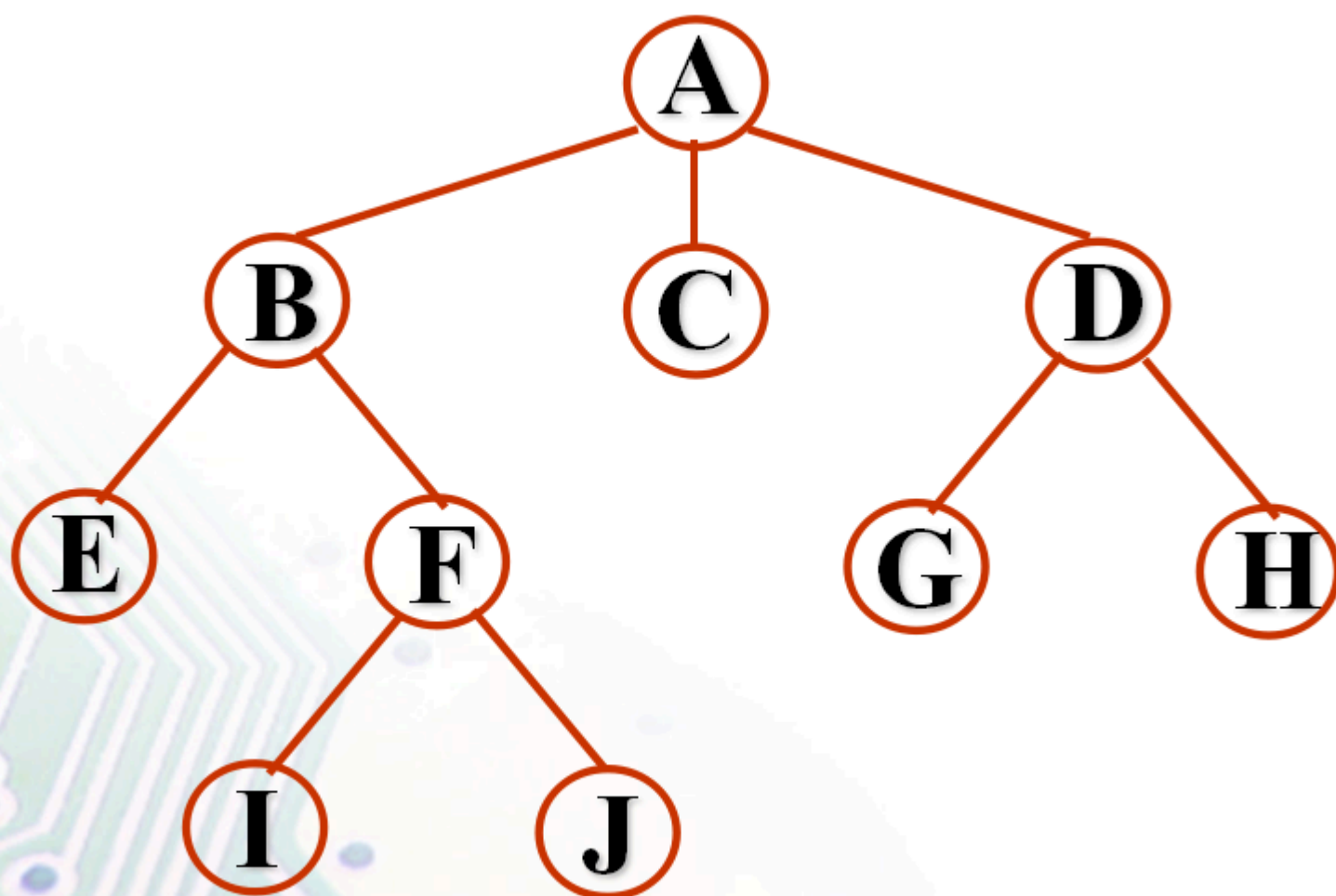
(subTree)。

- 每棵子树的根结点有且仅有一个直接前驱，但可以有0个或多个直接后继

几种表示方法

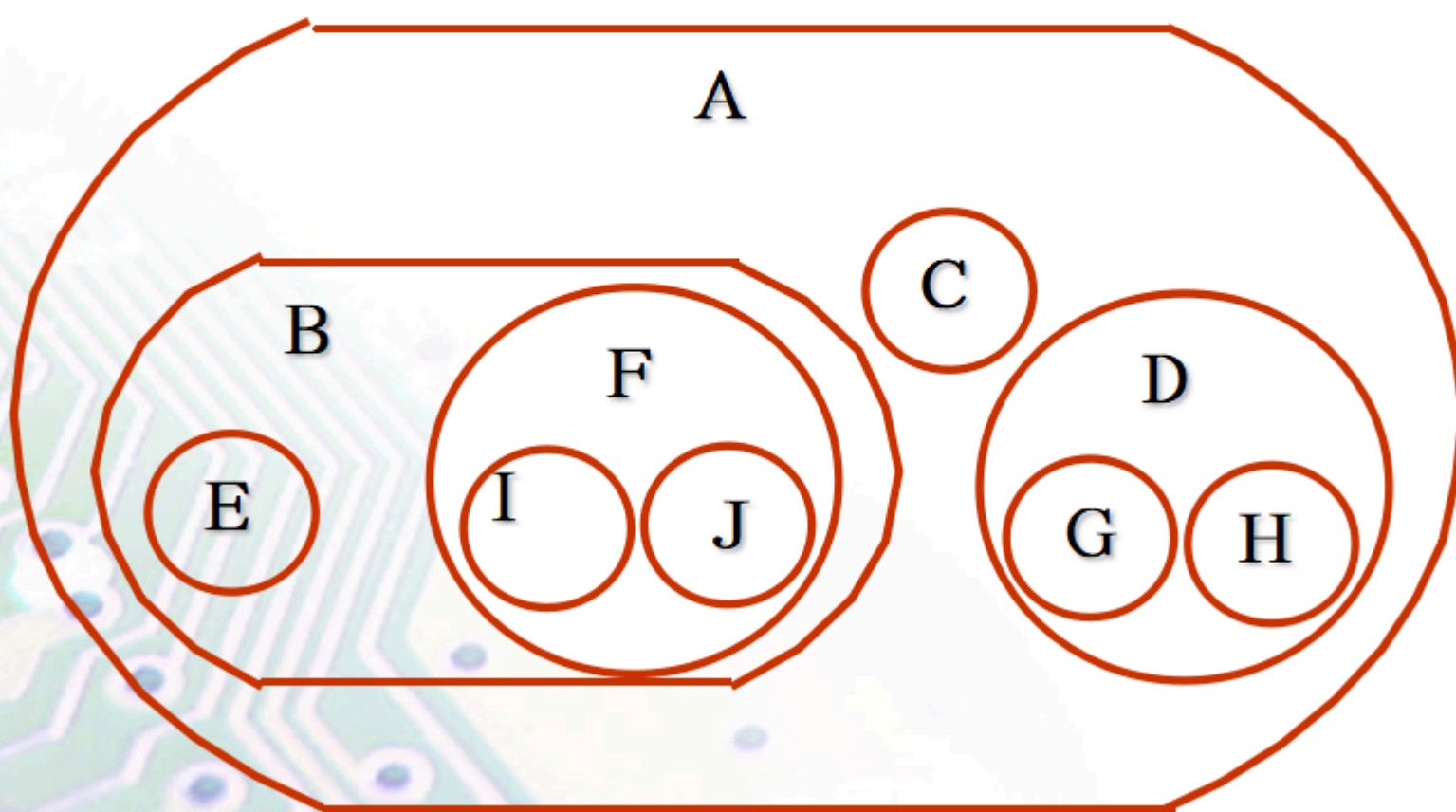
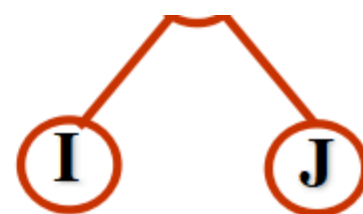
好像没什么用

👉 树形表示



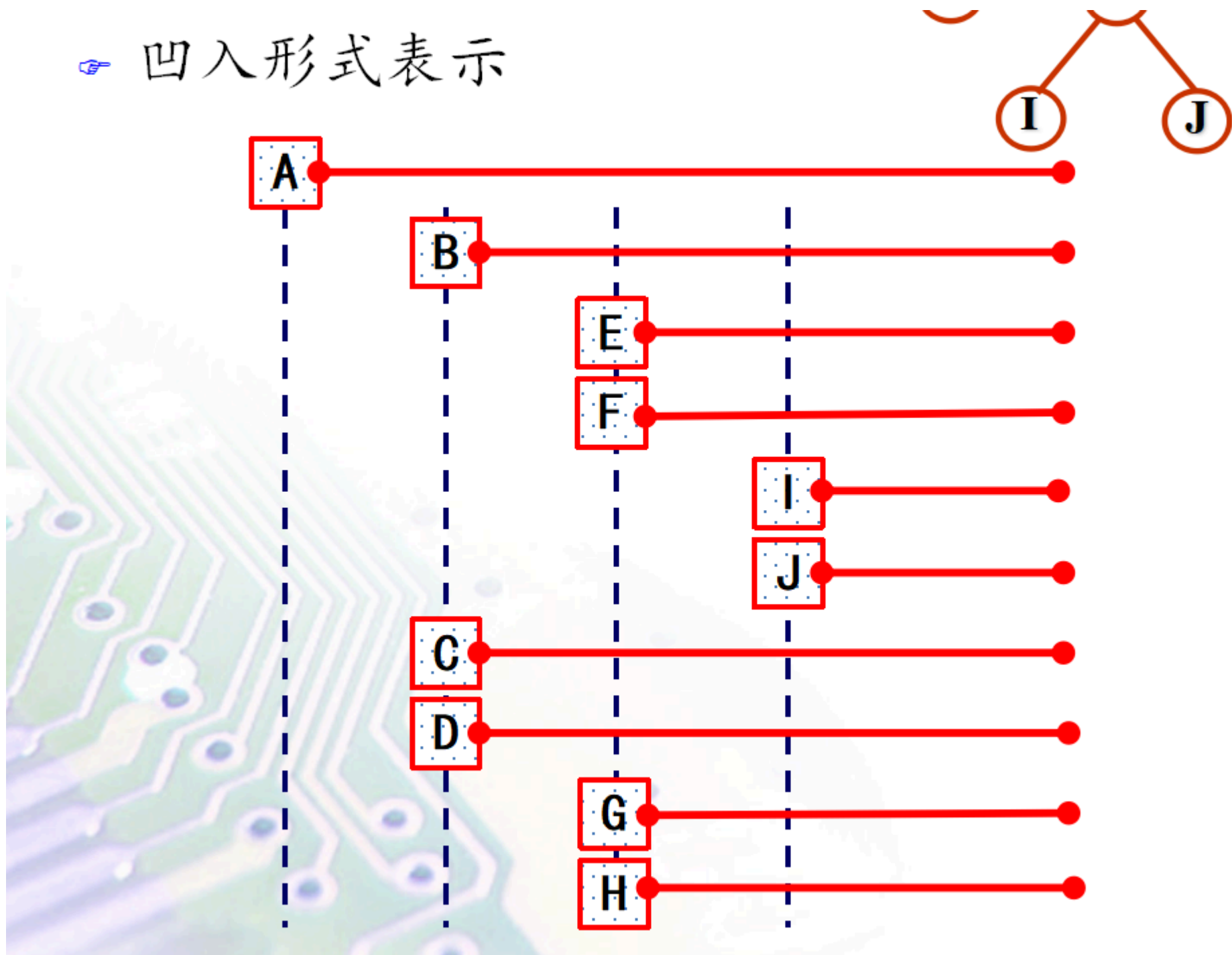
👉 嵌套集合（文氏图）表示

- 采用集合包含关系反映

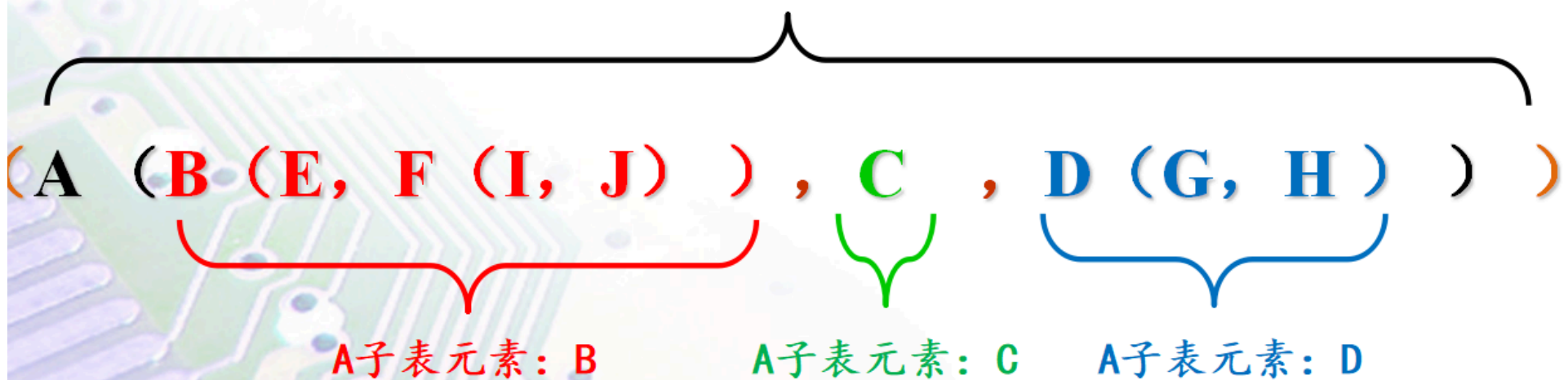


树的嵌套集合（文氏图）表示法

凹入形式表示



嵌套括号（广义表）包含一个元素（根）



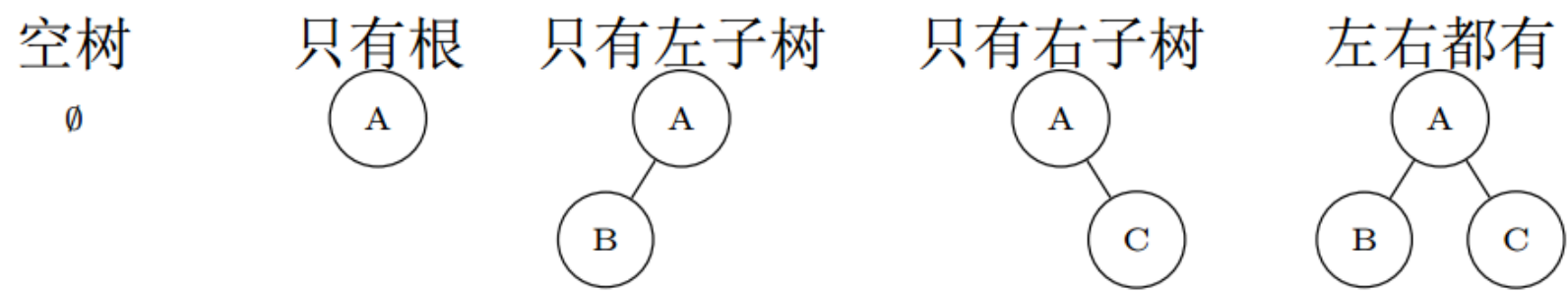
基本概念

- 节点的度：节点子树的个数
- 树的度：最大的节点度数
- 叶子节点：度为 0 的节点
- 分支节点：度不为 0 的节点
- 节点的层次：根为第 1 层，依次加 1
- 树的高度（深度）：最大层次数
- 孩子结点——某节点子树的根
- 双亲结点——相对于某节点子树的根
- 祖先节点——从根节点到该节点所经分支上的所有节点
- 子孙节点——某一节点子树中的所有节点
- 兄弟结点——同一双亲节点孩子节点互为兄弟节点

二叉树

二叉树是n(n≥0)个结点的有限集。它或者是空集(n=0)，或者由一个根结点及两棵互不相交的、分别称作这个根的左子树和右子树的二叉树组成

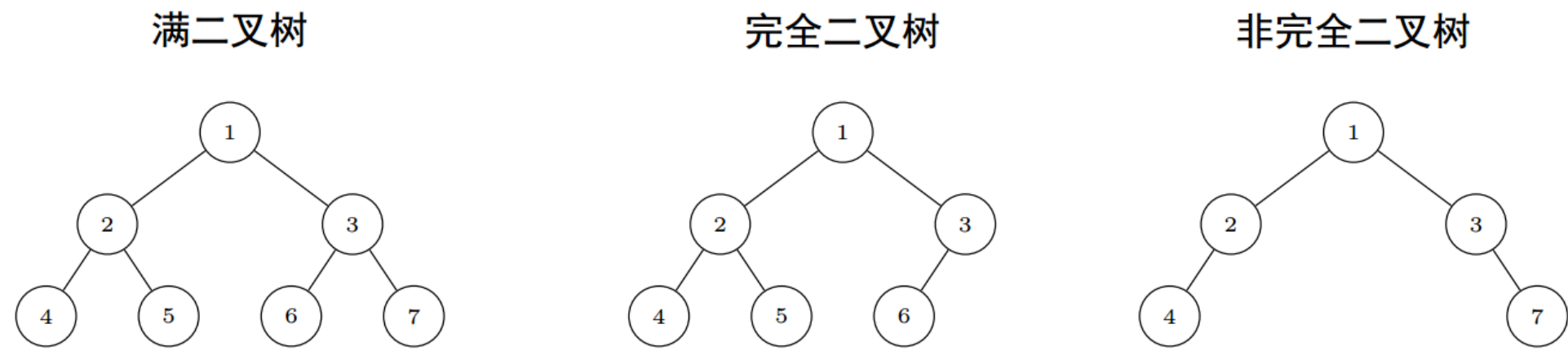
- 二叉树每个节点最多有两个子树
- 二叉树子树有左右之分，即使只有一个子树也要区分左右
 - 因此，二叉树是不同于树的东西，**二叉树不是树的特例**
- 五种基本形态：



- 对非空二叉树,如果其叶结点个数为 n_0 ,度为2的非叶结点个数为 n_2 , 则有 $n_0 = n + 1$
- 具有n个结点的二叉树的高度为

$n + 1$ 或 $(n + 1)$

特殊形态的二叉树



满二叉树

- 每一层的节点数都达到最大值
- 高度为 h 的满二叉树有 $2^h - 1$ 个节点
- 所有叶子节点都在最后一层

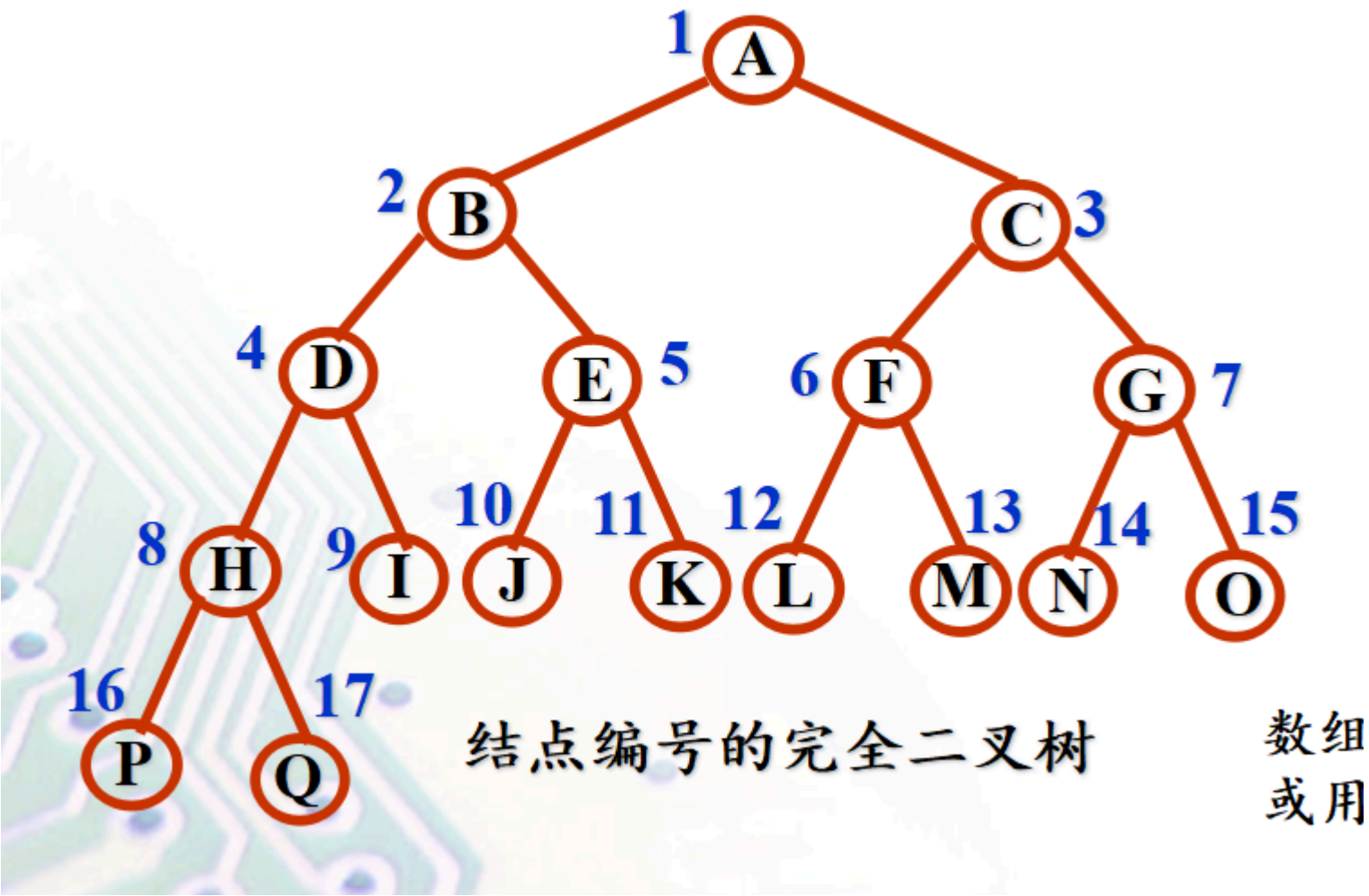
完全二叉树

- 除最后一层外，每层节点数都达到最大
- 最后一层的节点从左到右连续排列
- 可以看作从满二叉树的最下层从右向左删除若干叶子节点
- 满二叉树是完全二叉树，完全二叉树不一定是满二叉树

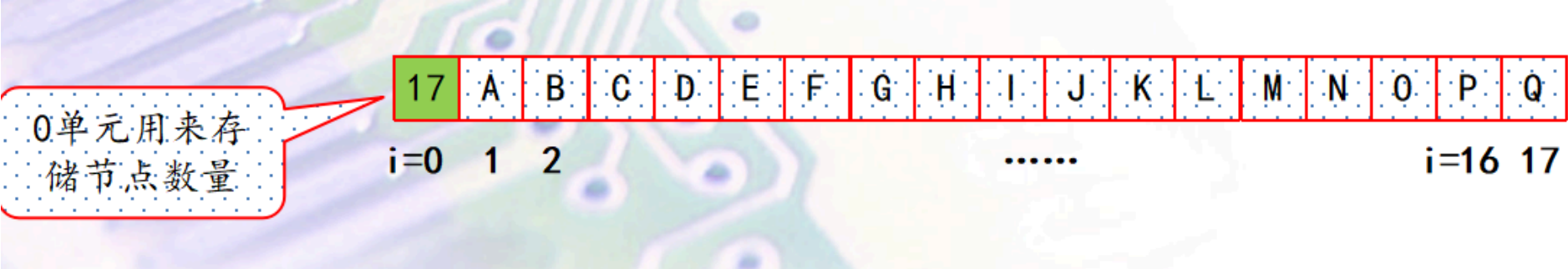
二叉树的存储

顺序存储

编号：



进行顺序存储：

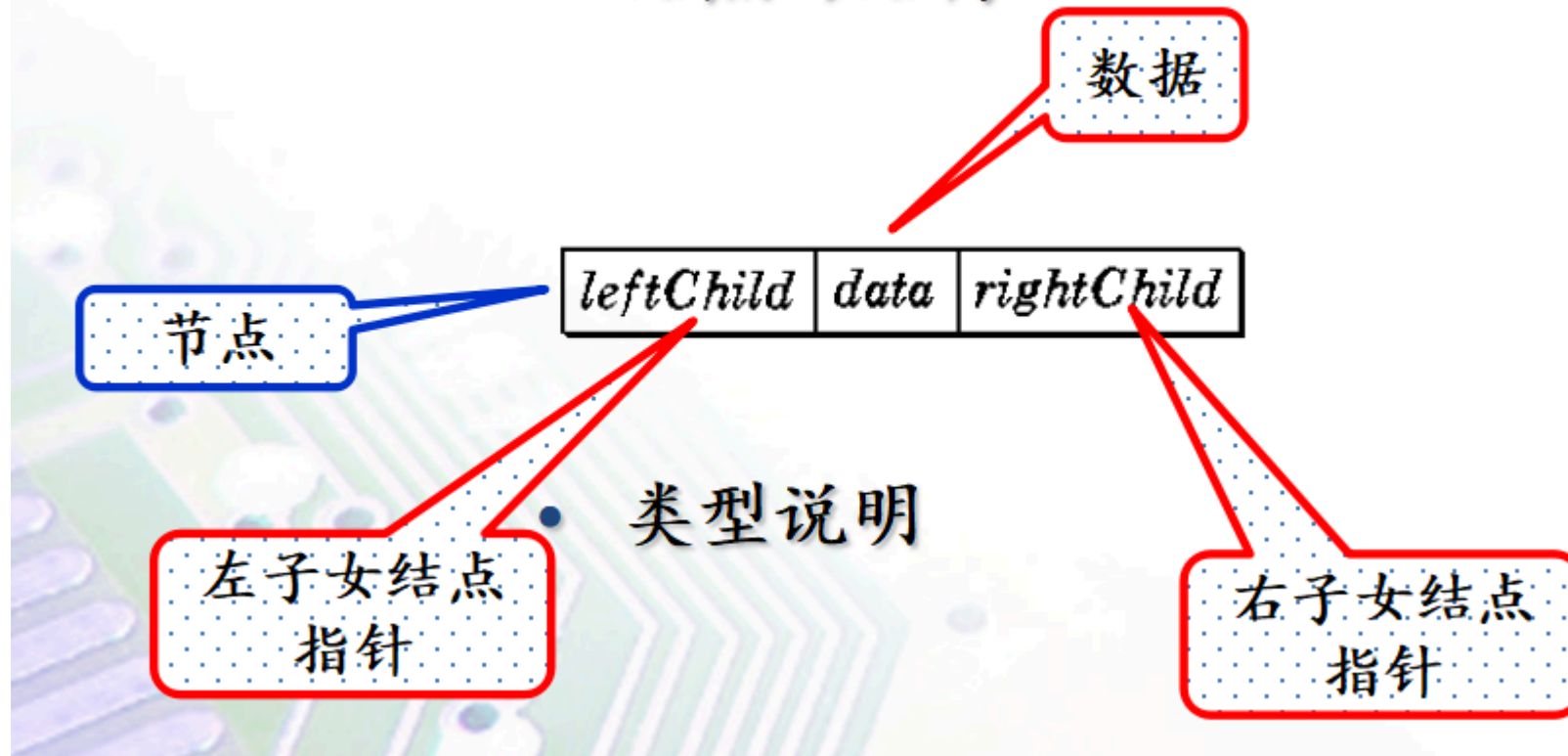


这种编号具有以下性质：

- 当 $i=1$ 时，该结点为二叉树的根;若 $i>1$ ，则该结点的双亲的序号为 i
- 若 $2i \leq n$ ，则该结点的左孩子的序号为 $2i$ ，否则该结点无左孩子
- 若 $2i+1 \leq n$ ，则该结点的右孩子的序号为 $2i+1$ ，否则该结点无右孩子

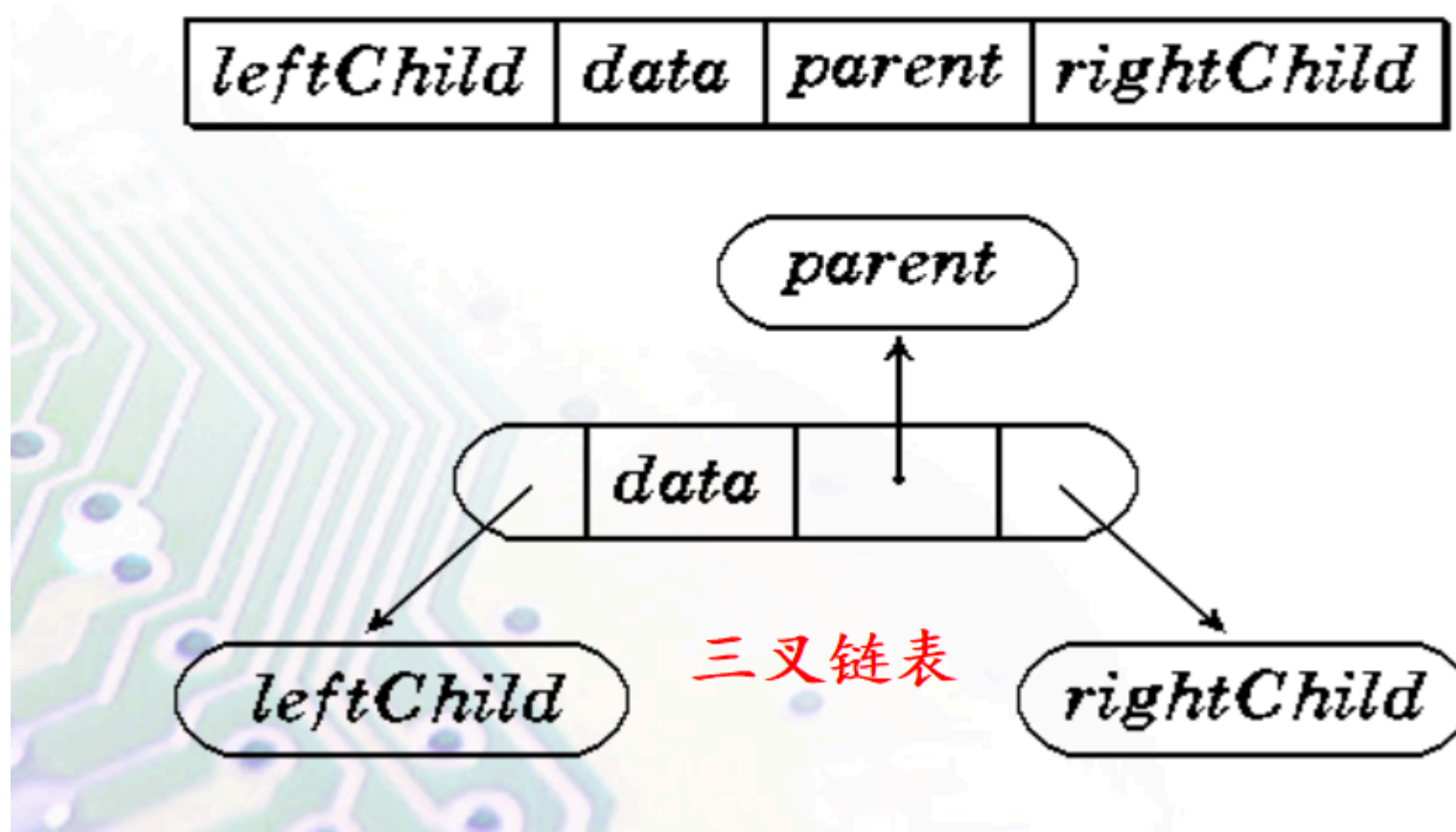
链式存储

- 结点的结构



- 类型说明

- 带双亲指针的二叉链表（三叉链表），略



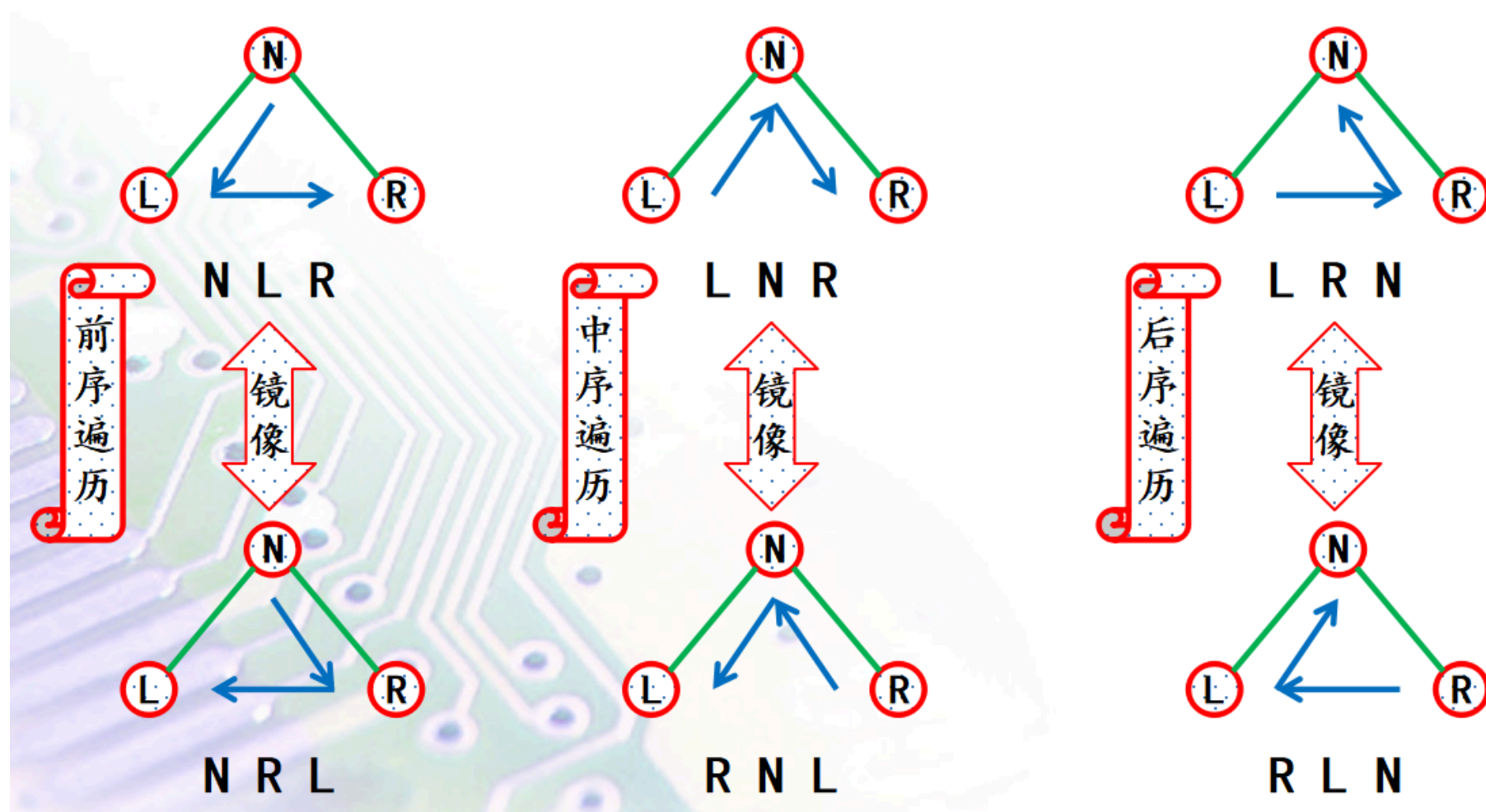
- n 个结点的二叉树中，共有 $n+1$ 个空链接域

| 二叉树的遍历

需求：访问节点本身、左子树、右子树

- 前序遍历
- 中序遍历

- 后序遍历



便捷方法：

[参考](#)

中序遍历的代码：

```
void InOrder(BinTree T){
    if(T){
        InOrder(T->lchild);
        printf("%c", T->data);
        InOrder(T->rchild);
    }
}
```

是否能从遍历结果逆推二叉树形态？

- 先序+中序→可以
- 后序+中序→可以
- 先序+后序→不可以

👉 先（后）序+中序逆推方法

- 1. 从先（后）序中确定根节点R(标记到稿纸上)
- 2. 在中序中确定左右子树L和R（标记到边上）
- 3. 在L和R中重复1-2步，直到L和R为空

| 森林

森林是树的集合

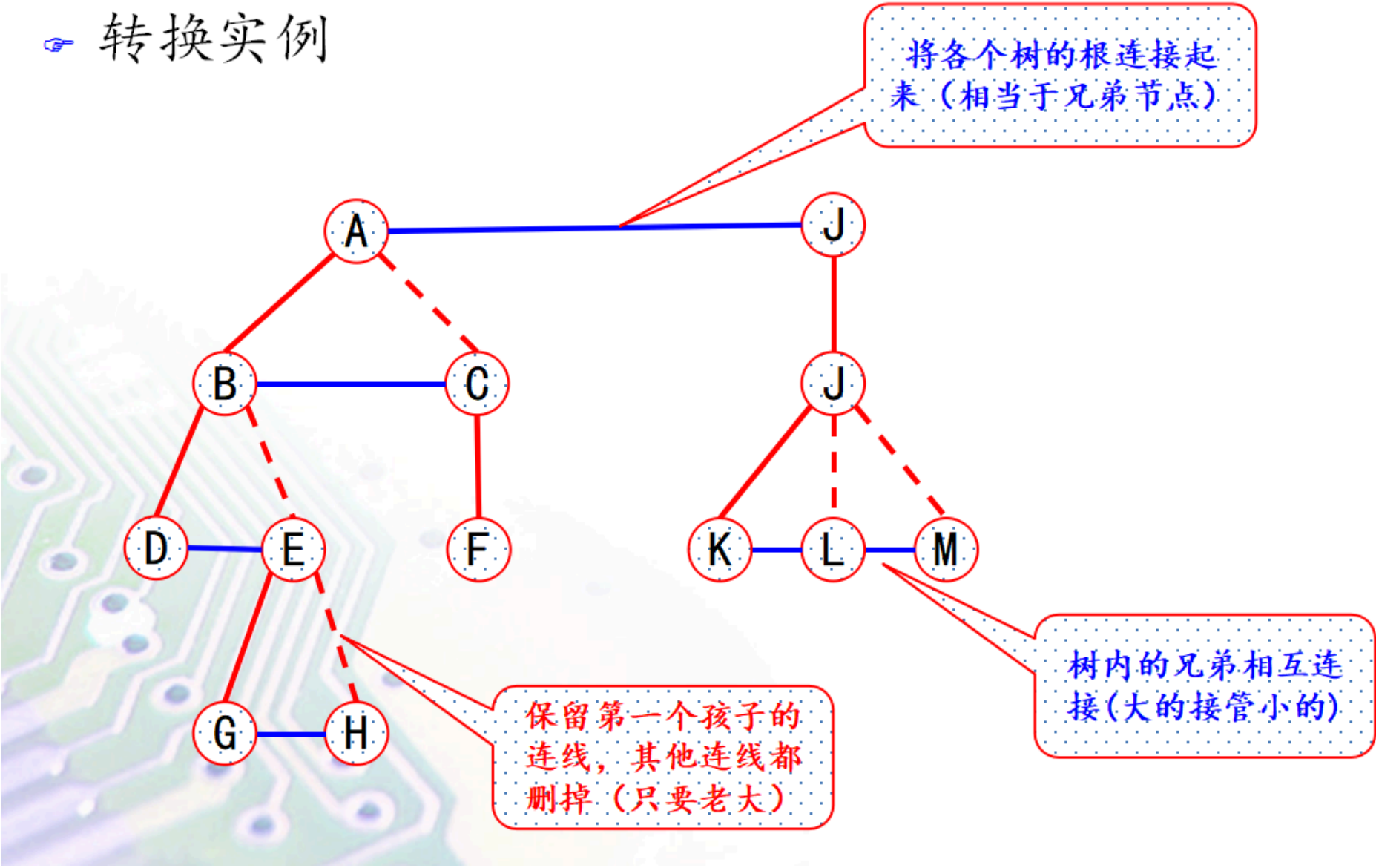
| 森林转二叉树

孩子→左，兄弟→右

1. 将每棵树转换为二叉树（第一个孩子作为左子树，兄弟作为右子树）
2. 将各树的根节点视为兄弟，连接起来

3. 第一棵树的根作为整个二叉树的根

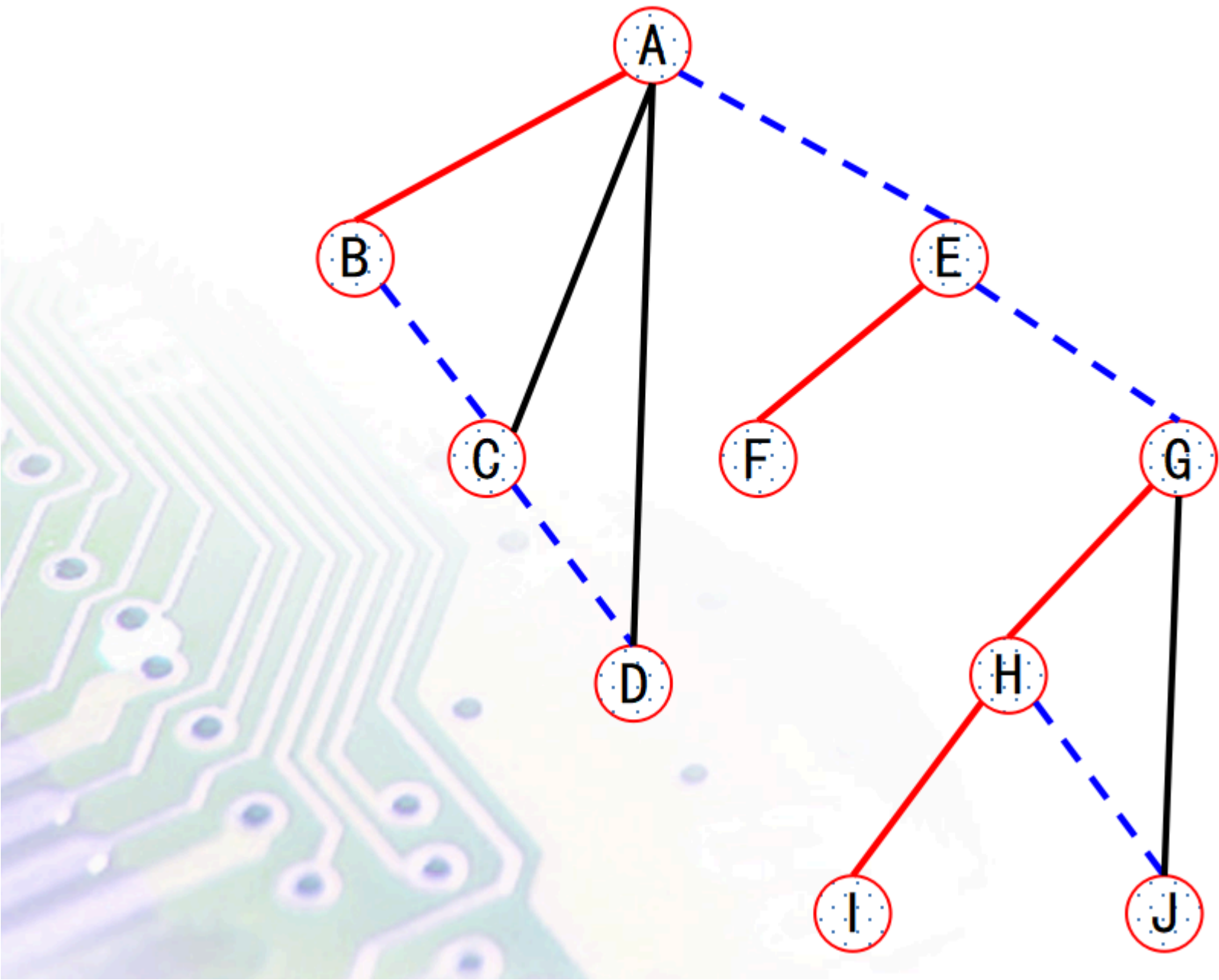
转换实例



二叉树转森林

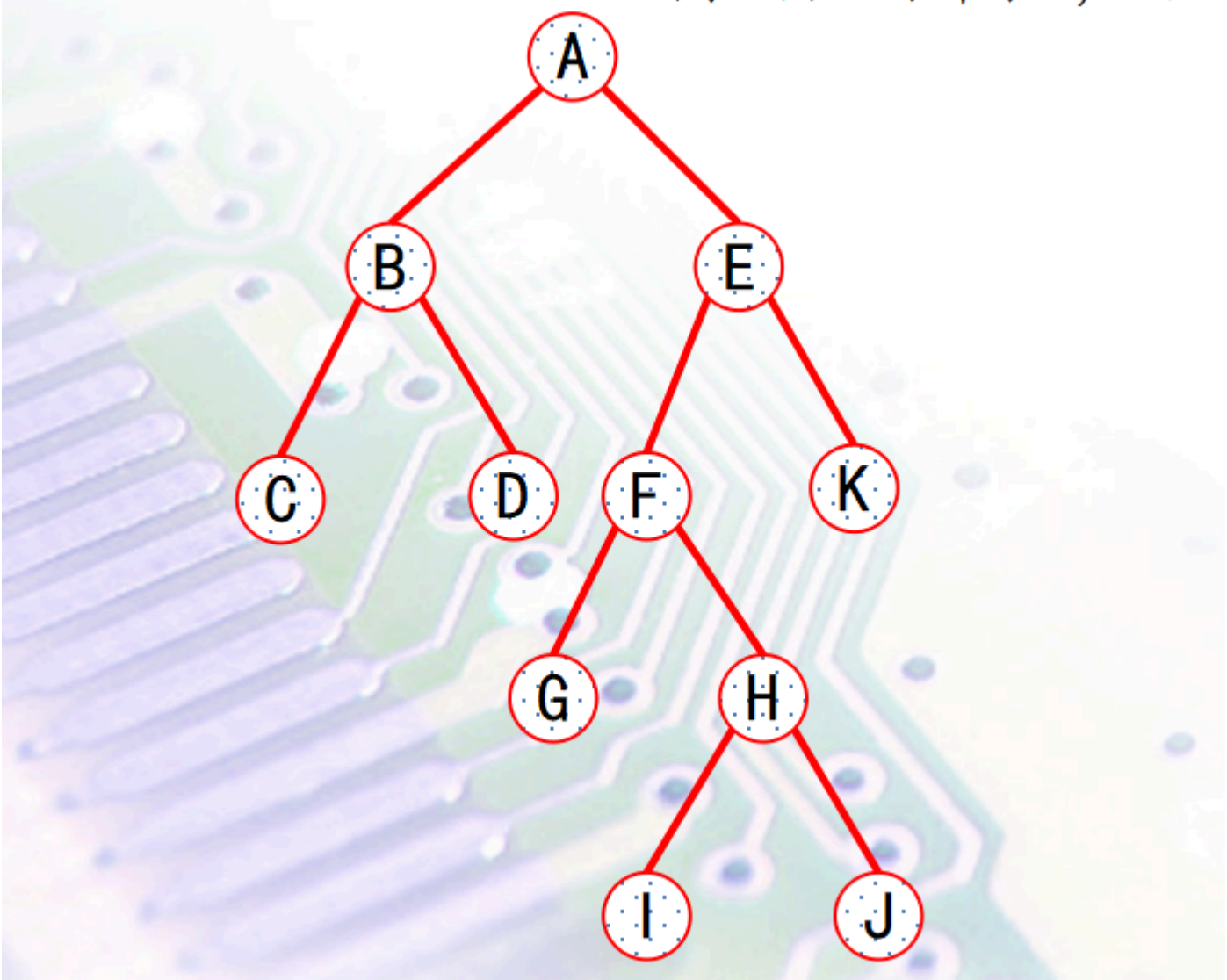
左→孩子, 右→兄弟

转换实例



霍夫曼树、霍夫曼编码

- 扩充二叉树
 - 指除叶子结点外，其余结点都必须有两个孩子的二叉树（任何节点，不能有1度，要么0度，要么2度）



- 内路径长度
 - 从根到所有非叶节点的路径长度之和
- 外路径长度
 - 从根到所有叶子节点的路径长度之和
- 加权路径长度
 - 各叶结点所带的权值与该叶结点到根的路径长度 i 的乘积之和

- $$L = \sum_{i=1}^n i * w_i$$

- 二叉树的带权路径长度WPL最小的称为**最优二叉树**，也称为**霍夫曼树**

| 霍夫曼算法

由n个只有根节点的树的集合（森林）构成霍夫曼树的过程

步骤：

1. 将 n 个权值构成 n 棵只有根的树的森林
2. 选取权值最小的两棵树合并，新树根的权值为两子树权值之和
3. 删除原来两棵树，将新树加入森林
4. 重复步骤 2-3，直到森林中只剩一棵树

| 霍夫曼编码

短码不是长码的前缀的编码称为“**前缀码**”

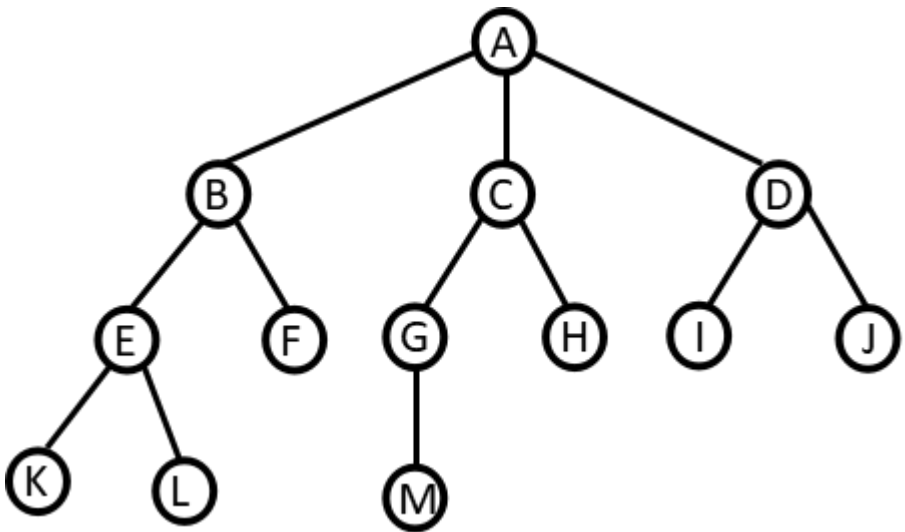
霍夫曼编码的特点

- 电文的总长度最短
- 是前缀码（译码唯一性，译码无歧义）

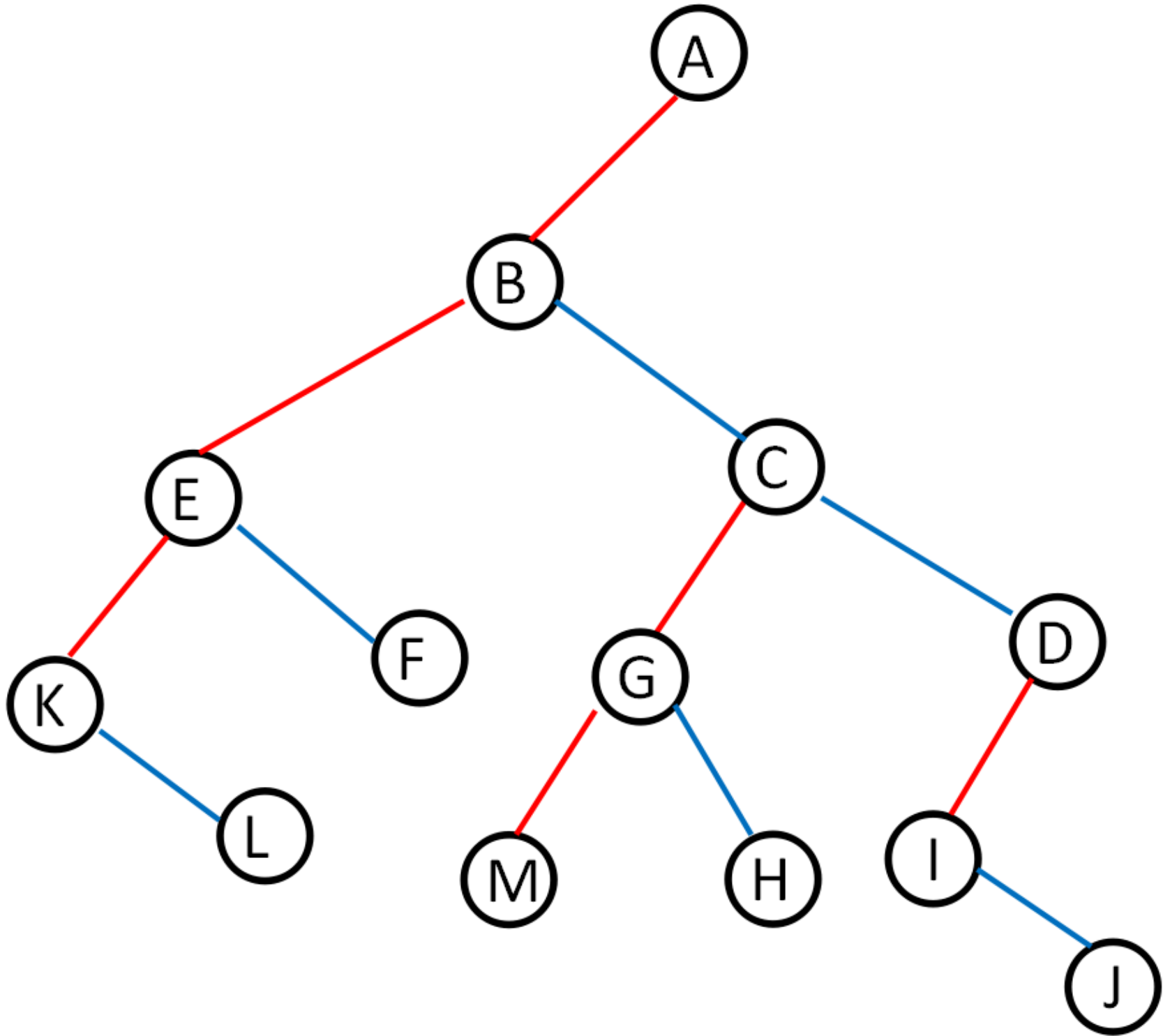
编码方式：

霍夫曼树中，左子树给一个0,右子树给一个1,总根节点到任一个节点总的编码就是霍夫曼编码

| 作业



画出该树等效的二叉树，并写出其先序、中序和后续遍历结果。

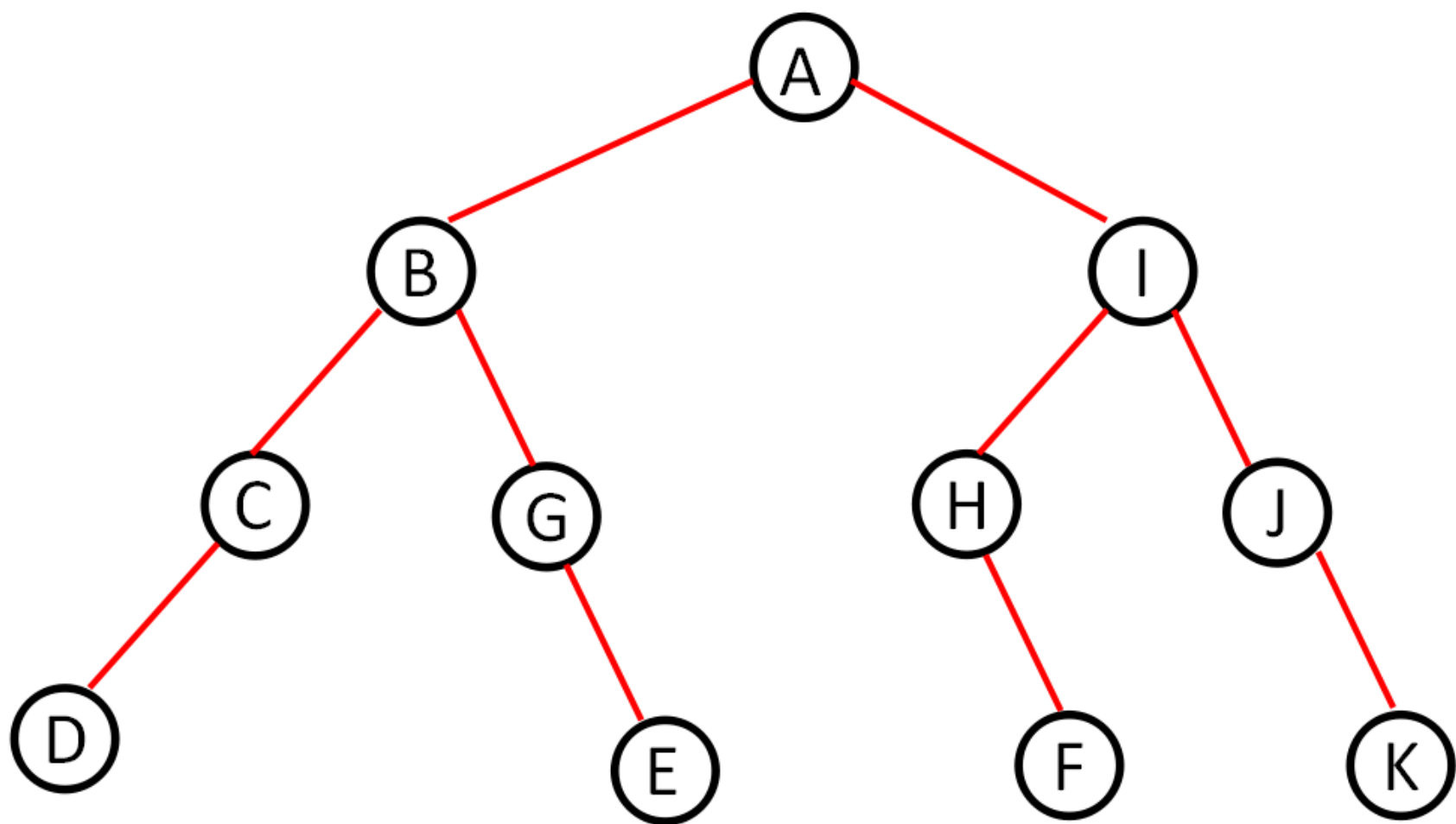


先序	A	B	E	K	L	F	C	G	M	H	D	I	J
中序	K	L	E	F	B	M	G	H	C	I	J	D	A
后序	L	K	F	E	M	H	G	J	I	D	C	B	A

先序	A	B	E	K	L	F	C	G	M	H	D	I	J
中序	K	L	E	F	B	M	G	H	C	I	J	D	A
后序	L	K	F	E	M	H	G	J	I	D	C	B	A

一棵二叉树的中序、后序序列如下，请画出该二叉树。

中序：DCBGEAHFIJK后序：DCEGBFHKJIA



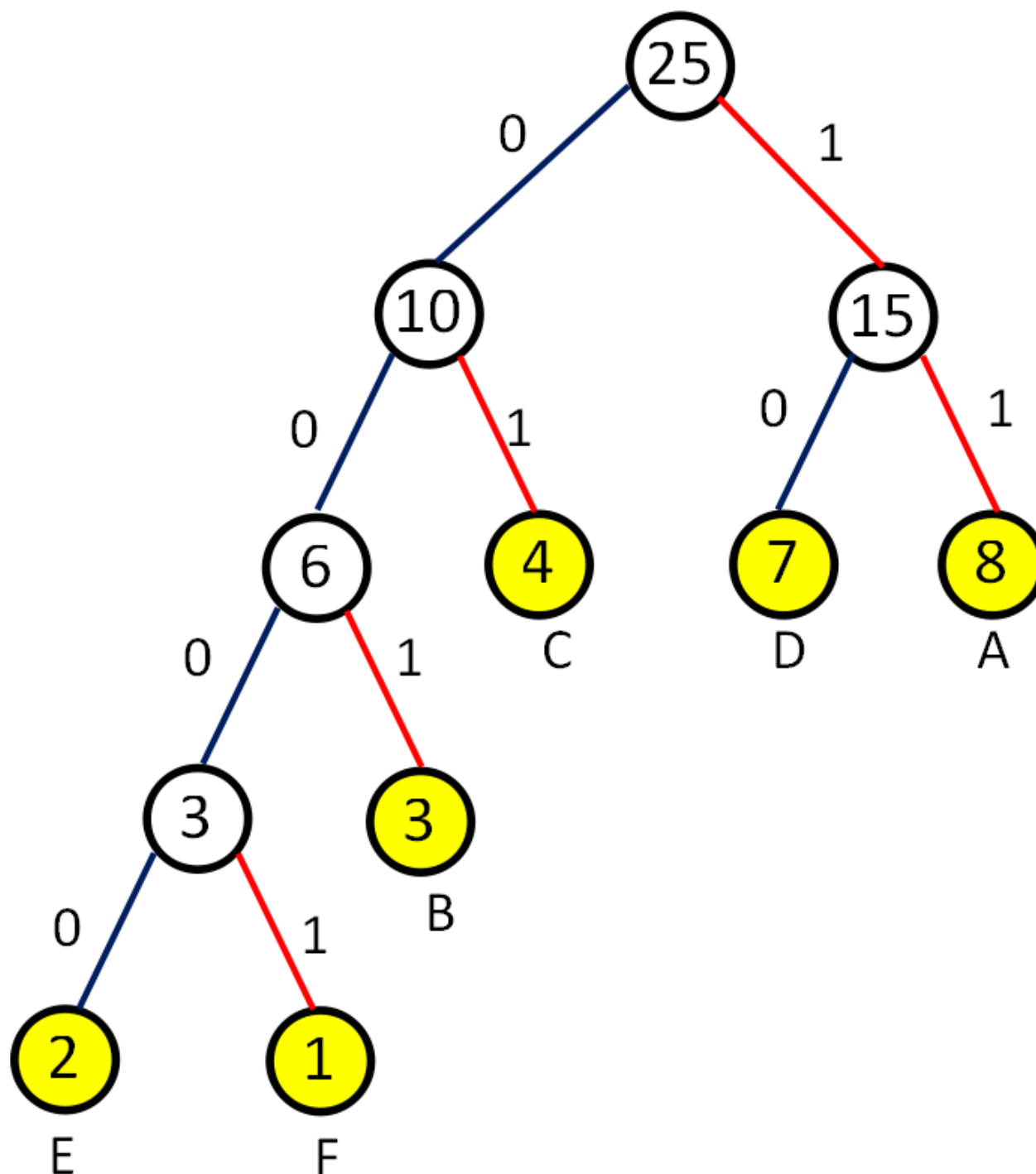
3

给定25个字符组成的电文为DDDDAAABEEAAFCDAABCCCBADD，请基于Huffman树为字符A、B、C、D、E、F设计Huffman编码。

各个字符出现的次数为：

A：8次，B：3次，C：4次，D：7次，E：2次，F：1次

设计对应的Huffman树：



对应的编码为：

A: 11
B: 001
C: 01
D: 10
E: 0000
F: 0001